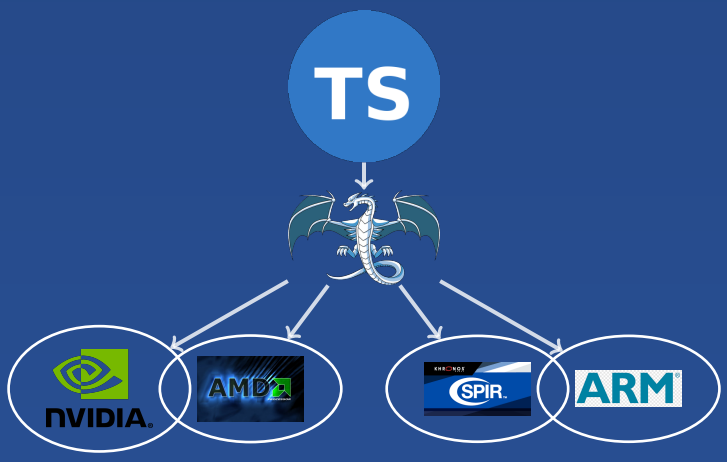
Velislav S. Karastoychev

Programming LLVM IR with TypeScript

A Practical Handbook for Cross-Platform and Multi-Architecture Code Generation



Programming LLVM IR with TypeScript

A Practical Handbook for Cross-Platform and Multi-Architecture Code Generation

 Velislav Karastoychev

Founder of Euriklis LTD
Co-founder of One Agent LTD

vskarastoychev@gmail.com

https://github.com/VelislavKarastoychev/euriklis-llvm-ir

January 11, 2026

Preface

The purpose of this handbook is to provide a comprehensive explanation of how to use the `@euriklis/llvm-ir` library for programmatic generation of LLVM intermediate representation targeting CPUs and GPUs from TypeScript and JavaScript.

We have chosen a pedagogical approach that emphasizes **learning through examples**. This methodology is particularly well-suited for a handbook intended as a practical guide. Rather than starting with abstract theory, we demonstrate concepts through working code that readers can immediately understand, modify, and experiment with. Each example builds upon previous ones, gradually introducing complexity while maintaining clarity.

However, given the inherent theoretical complexity of LLVM IR, compiler design, and heterogeneous computing, we also provide comprehensive theoretical foundations. The handbook includes formal LLVM definitions, detailed explanations of all instructions in the intermediate language, memory models, type systems, and architecture-specific intrinsics. This dual approach—practical examples supported by rigorous theory—ensures the handbook serves both as a tutorial for beginners and as a reference for experienced developers.

Acknowledgments

I would like to thank my friend **Aristeidis Georgakis** for always being there for me in my most difficult moments—morally, materially, and scientifically.

I would also like to express my deep gratitude to my friend **Michail Urilski**, who for years has listened to my "nonsense" and, despite everything, has not given up on life. Without his financial support, my article might not have been published.

I would like to express my deep gratitude to ***Dr. Peter Sveshtarov*** for the fruitful discussions and guidance.

List of Abbreviations

ABI	Application Binary Interface
ACM	Association for Computing Machinery
Ada	Programming language developed by U.S. Department of Defense
AI	Artificial Intelligence
AIR	Apple Intermediate Representation
Algol	ALGOrithmic Language
AMD	Advanced Micro Devices
AMDGPU	AMD Graphics Processing Unit
API	Application Programming Interface
ARM	Advanced RISC Machine
AST	Abstract Syntax Tree
AVX	Advanced Vector Extensions
AWS	Amazon Web Services
BASIC	Beginner's All-purpose Symbolic Instruction Code
CDNA	Compute DNA (AMD GPU architecture)
CFG	Context-Free Grammar
COBOL	Common Business-Oriented Language
CSV	Comma-Separated Values
CPU	Central Processing Unit
CSE	Common Subexpression Elimination
CUDA	Compute Unified Device Architecture
DRAM	Dynamic Random Access Memory
DSL	Domain-Specific Language
DFA	Deterministic Finite Automaton
ELF	Executable and Linkable Format

ETL	Extract, Transform, Load
FFI	Foreign Function Interface
FFT	Fast Fourier Transform
FIRST	First Set (parser lookahead)
Fortran	FORmula TRANslation
GCN	Graphics Core Next (AMD GPU instruction set)
GEP	GetElementPtr (LLVM instruction)
GCC	GNU Compiler Collection
GNU	GNU's Not Unix
GPU	Graphics Processing Unit
HIP	Heterogeneous-Compute Interface for Portability
HSA	Heterogeneous System Architecture
HSACO	HSA Code Object
IDE	Integrated Development Environment
IoT	Internet of Things
IPO	Inter-Procedural Optimization
IR	Intermediate Representation
ISA	Instruction Set Architecture
JIT	Just-In-Time compiler
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LALR	Look-Ahead LR parser
LDS	Local Data Share (AMD GPU memory)
LLVM	Low Level Virtual Machine
LL	Left-to-right scan, Leftmost derivation parser
LR	Left-to-right scan, Rightmost derivation parser
MATLAB	MATrix LABoratory
MIPS	Microprocessor without Interlocked Pipelined Stages
ML	Machine Learning
MLIR	Multi-Level Intermediate Representation
MPS	Metal Performance Shaders

MPSGraph	Metal Performance Shaders Graph
NEON	ARM SIMD instruction set
NFA	Nondeterministic Finite Automaton
NPM	Node Package Manager
NVPTX	NVIDIA Parallel Thread Execution
NVVM	NVIDIA Virtual Machine
OpenCL	Open Computing Language
PC	Personal Computer
PHI	Phi instruction (from φ in SSA form)
PTX	Parallel Thread Execution
RDNA	Radeon DNA (AMD GPU architecture)
RISC	Reduced Instruction Set Computer
RISC-V	Reduced Instruction Set Computer - Version 5
ROCm	Radeon Open Compute platform
RTX	Ray Tracing Texel eXtreme (NVIDIA)
SASS	Shader Assembly (NVIDIA native GPU assembly)
SIMD	Single Instruction Multiple Data
SLR	Simple LR parser
SPIR	Standard Portable Intermediate Representation
SPIR-V	Standard Portable Intermediate Representation - Vulkan
SQL	Structured Query Language
SRAM	Static Random Access Memory
SSA	Static Single Assignment
SSE	Streaming SIMD Extensions
SYCL	Standard C++ parallel programming framework
VHDL	VHSIC Hardware Description Language
VRAM	Video Random Access Memory
Vulkan	Cross-platform graphics and compute API
WASM	WebAssembly
WebGPU	Web Graphics Processing Unit API
x86	Intel/AMD processor architecture family (originated from Intel 8086 processor)
x86-64	64-bit extension of x86 architecture (also known as AMD64 or Intel 64)

Contents

Part I

Introduction

Chapter 1

What is LLVM IR and Why Is It Useful?

1.1 The Post-PC Era and New Computational Requirements

The transformation of human labor practices presents qualitatively new challenges for society to address. Computing technologies, as an indispensable instrument for solving most of these challenges, must also adapt appropriately to meet the needs of emerging economic and social realities. Moreover, it appears that changes in computing technologies, like socio-economic transformations, follow the principles of evolution and co-evolution. This perspective is more than mere metaphor—researchers such as Leo Meyerovich and Ariel Rabkin [14] document how programming languages evolve under selective pressure from hardware capabilities, business requirements, and competing technologies, much like biological species in a changing environment.

Indeed, the first computers were programmed in machine code. Later, for convenience, machine instructions were replaced with mnemonics that represented a bijective correspondence to them. Machine code and mnemonic languages (assembly) are, in their instruction semantics, much closer to programming physical memory and computer hardware, making them unsuitable for programming abstract processes. This inconvenience gave rise to the need for a more abstract ISA, enabling the description of processes of broader scientific interest.

Since the first scientific communities to adopt computing technologies were mathematicians and engineers, it was natural to expect that the first programming languages beyond mnemonics would have ISAs particularly suited for developing and programming algorithms and mathematical computations. Thus, the first high-level languages such as Fortran (1957) and Algol (1958) were created primarily for building algorithms and mathematical computations. They employed ISAs close to mathematical notation, which through parsing were "translated"—or as the established term goes, "compiled"—to assembly, and from there to machine code.

However, these languages proved less suitable for programming business and production tasks (writing operating systems, inventory management programs, reading data from multiple inputs, etc.). The search by programmers for a suitable language during this period did not follow any particular pattern but rather had a somewhat random character. For example, Pascal was developed by Niklaus Wirth in 1970 to teach students structured programming, Ada was developed by the U.S. Department of Defense in 1980 for critical military and aviation systems with high reliability and safety requirements, and BASIC was developed by John Kemeny and Thomas Kurtz in 1964 at Dartmouth College as a simple language for teaching novice programmers.

The most successful proved to be the C programming language, created by Dennis Ritchie at Bell Labs in 1972. This language possessed the best combination of ISAs suitable for both algorithmic and hardware design. With it, one could conveniently write both algorithmically formulated problems and system programming tasks (such as writing an operating system—indeed,

the C compiler is written in C itself, the so-called "self-hosting compiler"). Additionally, it was sufficiently efficient in terms of performance. Here again, the compiler played a crucial role, as its task was to reduce C instructions to assembly instructions.

Thanks to the C language, many different types of languages were created, such as Prolog (1972), C++ (1985), Perl (1987), Python (1991), Ruby (1995), and many others. Some languages resembled C/C++ in syntax and functionality, while others sought to differentiate themselves from its semantics.

Particularly noteworthy is the Java language (created by James Gosling at Sun Microsystems in 1995), which resembles C++ but compiles not to machine code but to bytecode, which is then executed by the JVM for each computer architecture, making it maximally portable ("write once, run anywhere"). Additionally, Java does not allow direct memory management (automatic garbage collector).

Other languages diverged from the C paradigm, such as JavaScript (created by Brendan Eich at Netscape in 1995 in 10 days) and Python. They completely eliminated strict static typing of data and focused on ISAs optimized for rapid prototyping, dynamic data processing, and interactive development.

Gradually, languages that do not use strict static data types and have simplified syntax with limited or no direct machine instructions (high level of abstraction) gained prominence in practice. The nature of software products established the paradigm of object-oriented programming over others. If the path of programming were linear or governed by some law, one would expect these trends to solidify over time. However, precisely the opposite occurred.

After the establishment of web technologies and mobile platforms (the "Post-PC" era after 2010), the need for more efficient process execution brought strictly typed languages back into fashion, with Java and C# becoming most prevalent. JavaScript responded to this challenge by becoming a backend language through Node.js technology (created by Ryan Dahl in 2009, later also through Deno and Bun) and through its "pseudo-typing" in TypeScript (created by Microsoft in 2012, adding static types to JavaScript).

Python responded through the use of JIT compilers like Numba or PyPy, as well as through new compilers like Cython, while also introducing optional typing (type hints) from version 3.5 (2015).

But the story does not end here. Writing increasingly efficient programs that process big data becomes an ever-growing challenge, and one of the simplest working solutions is parallel processing [17]. Initially, parallel processing was reduced to multi-threading, later expanding to interconnected computing machines (cluster computing, distributed computing), and today the paradigm of heterogeneous programming has emerged, where program execution is distributed between the CPU and the GPU.

This led once again to the "reign" of C (and C++), which supports GPU programming through technologies like CUDA, OpenCL, SYCL, and protocols for managing GPU memory, with strictly typed data being a mandatory requirement for executing code on GPUs (due to hardware-level optimizations and SIMD).

Languages like Python find solutions to this problem by creating compilers for heterogeneous programs using LLVM (such as Numba for CUDA kernels), while others like TypeScript use FFI protocols to call C/C++ code. It should be noted that all languages use FFI protocols or similar mechanisms, since only C/C++ (and other systems languages) can directly manage GPU memory and invoke CUDA/OpenCL/Vulkan APIs.

The attentive reader will easily notice that in this case as well, the development of programming languages does not follow some strictly defined linear path but is a series of contingencies, societal needs, and reserves of variability that languages possess to respond to the demands of emerging environments. Thus, the trajectory of programming language development has a rather zigzag pattern and is evolutionary in nature, not deterministic. This dynamic resembles the Red Queen Hypothesis from evolutionary biology—programming languages must constantly adapt

not to displace a given paradigm or technology, but to remain competitive in a rapidly changing technological environment. Conversely, the adaptation of individual languages and computing technologies changes the technological environment itself.

Thus, already-created programming languages do not disappear but, through the forces of interdependence and functional complementarity, specialize in different niches, such as scientific (Fortran, MATLAB, Wolfram Language, R), DSL (SQL for databases, Verilog/VHDL for hardware design, shader languages for GPU graphics), and legacy niches (COBOL for financial infrastructure, Ada for aviation safety, Fortran for climate models). The factor that determines how a programming language or computing technology will situate itself relative to others is the capacity for maximal adaptation to the environment and the specialization niche.

Another phenomenon worthy of note is that once again there arises the need for compilers and compilation infrastructure (such as LLVM) to optimize code efficiency for different architectures (x86, ARM, RISC-V, GPU). LLVM serves as a "universal adapter" in this ecosystem, allowing different languages to compile efficiently for multiple hardware platforms—yet another example of technological co-evolution.

This understanding of computing technologies and interpretation of events in the programming sphere I largely obtained from the prominent scientist, aristocrat, and public benefactor Peter Sveshtarov through the numerous conversations we have had. Thanks to him, I was inspired to build this project.

1.2 The Compiler Problem and LLVM's Solution

As we have seen, the evolutionary trajectory of programming languages has led to an ecosystem where multiple languages coexist, each occupying its specialized niche. However, this diversity creates a fundamental engineering challenge: the compiler multiplication problem.

According to the Turing model of computation, every programming language can be formalized as a finite set of instructions (or operations) designed to perform five fundamental categories of operations: **(1) memory read operations**, **(2) memory write operations**, **(3) arithmetic and logical computations**, **(4) conditional evaluation** (testing the truth value of logical predicates), and **(5) control flow operations** (loops, branches, and jumps). This theoretical framework establishes that any computable function can be expressed through combinations of these primitive operations.

However, physical computing hardware—specifically the **central processing unit (CPU)** or **microprocessor**—can execute only a limited set of primitive operations implemented in hardware logic gates. At the lowest level, processors operate on binary representations through **Boolean logic operations** (bitwise AND, OR, XOR, NOT) and basic arithmetic operations implemented in silicon. Every CPU architecture (x86-64, ARM, RISC-V, etc.) defines its own **ISA**—a specification of the machine instructions that the processor can execute directly in hardware. These machine instructions are encoded in binary format (machine code) and can be represented in human-readable form using **assembly language**, where each assembly instruction corresponds to a specific machine opcode.

The assembly language is architecture-specific: x86-64 assembly differs fundamentally from ARM assembly, both in syntax and available instructions. A microcontroller designed for embedded systems may support only a minimal ISA, while a modern GPU implements a vastly different instruction set optimized for parallel computation.

Since high-level programming languages (C, TypeScript, Python, etc.) provide abstractions far removed from machine instructions, there must exist a mechanism to translate source code into executable machine code for the target architecture. This translation is performed by specialized software systems:

1.2.1 Compiler

A compiler translates the entire source program into native machine code (or assembly) *before execution*. The canonical compiler architecture, established by Aho, Sethi, Ullman, and Lam [2] in their seminal work *Compilers: Principles, Techniques, and Tools* (the "Dragon Book"), organizes compilation into distinct phases:

1. **Lexical Analysis (Scanning):** The source code character stream is partitioned into lexical units called *tokens*. Aho et al. formalize this process using *regular expressions* and *finite automata*[1] (both deterministic and nondeterministic). Cooper and Torczon [12] emphasize that modern scanners employ *maximal munch*[2] strategies and *DFA minimization* for efficiency.[3] Appel [3] provides practical implementations demonstrating how tokens are recognized through state transitions in finite automata. Recent algorithmic improvements [1] achieve linear-time tokenization without excessive memory consumption.[4]

[1] A **finite automaton** is a mathematical model of computation that processes input character-by-character through a series of states. Think of it as a state machine that reads a string and decides whether it matches a pattern (like a regular expression). There are two types: **Nondeterministic Finite Automata (NFA)** allow multiple possible transitions for the same input character from a given state, or even transitions without consuming input (*epsilon transitions*). For example, when matching the pattern `/ab|ac/`, an NFA at state 0 reading 'a' could simultaneously explore both the 'ab' branch and the 'ac' branch. NFAs are conceptually simpler and correspond directly to regular expression syntax, but require backtracking or parallel state tracking during execution. **Deterministic Finite Automata (DFA)** have exactly one transition per input character from each state, guaranteeing predictable, linear-time execution without backtracking. The tradeoff is that DFAs can be exponentially larger than equivalent NFAs. JavaScript's `RegExp` engine internally uses similar automata-based algorithms, though modern engines employ optimizations like JIT compilation and backtracking for complex patterns. Compilers typically convert regular expressions to NFAs (easy to construct), then convert NFAs to DFAs (fast to execute), and finally minimize the DFA (reduce memory footprint) using algorithms like Hopcroft's algorithm [2].

[2] The **maximal munch** rule (also known as the *longest match* rule or *greedy matching*) is a disambiguation strategy where the lexer always consumes the longest possible sequence of characters that forms a valid token. For instance, when scanning `"123.45"`, maximal munch ensures recognition as a single floating-point literal rather than splitting it into separate tokens `"123"`, `"."`, and `"45"`. Similarly, `">="` is tokenized as a single greater-than-or-equal operator instead of `">"` followed by `"="`. This rule is essential for disambiguating cases like `"int"` (keyword) versus `"integer"` (identifier)—the scanner must consume the entire identifier rather than stopping prematurely at the keyword prefix.

[3] An alternative theoretical approach to pattern matching uses the **FFT (Fast Fourier Transform)**, which represents strings as polynomials and exploits polynomial multiplication via convolution to detect pattern occurrences. Fischer and Paterson [4] pioneered this technique, demonstrating that string matching is formally analogous to integer multiplication. By converting the pattern and text into polynomial representations and performing multiplication in the frequency domain, FFT-based algorithms achieve O(n log m) time complexity for text of length n and pattern of length m. While classical DFA-based lexers remain dominant in compiler construction due to their simplicity and predictable performance for typical programming language tokens, FFT-based approaches excel in specialized domains requiring *approximate pattern matching* (matching with mismatches) or *wildcard patterns* (don't-care symbols). Notable applications include: **SWIFT** (Sequence-Wide Investigation with Fourier Transform), a bioinformatics tool for protein classification from genome sequences; **agrep**, which performs approximate string matching using efficient algorithms suitable for DNA sequence analysis; and various computational biology tools that perform complex correlation-based matching on biological sequences in O(n log n) time. These FFT-based methods demonstrate the power of polynomial representations for pattern detection tasks where traditional automata approaches would require prohibitive memory or cannot handle approximate matching efficiently.

[4] Li and Mamouras [1] address the *uniform tokenization problem*—finding optimal algorithms that tokenize input text in linear time (proportional to input length) while avoiding the exponential memory consumption inherent in classical DFA-based approaches for complex grammars. They propose two novel algorithms: **(1)** the *token skipping* approach, which performs a right-to-left pass computing the "longest match" semantics over the reversed input string using the reversed tokenization grammar to construct a "tape" data structure where tape[i] stores the length of the maximal-munch token starting at position i. The subsequent left-to-right pass simply extracts tokens by jumping directly to boundaries encoded in the tape (e.g., first token is w[0..tape[0]], second token is w[tape[0]..tape[0]+tape[tape[0]]], etc.), completely eliminating backtracking. **(2)** The *token extension oracle* approach performs a right-to-left pass simulating the reverse tokenization NFA on the reversed input,

2. **Syntactic Analysis (Parsing):** Tokens are organized into a hierarchical *AST*[5] according to the language's *context-free grammar*[6]. The Dragon Book distinguishes between *top-down parsing*[7] (recursive descent, LL grammars) and *bottom-up parsing*[8] (LR, LALR, SLR parsers). Appel [3] advocates for *parser generators*[9] (YACC, Bison) that automatically construct parsers from grammar specifications. Cooper and Torczon [12] highlight that modern parsers must handle *ambiguous grammars*[10] through precedence and associativity declarations, and they emphasize *error recovery*

recording configurations at each position to create an oracle. During the subsequent left-to-right pass, when a final state is reached (indicating a potential token), the oracle determines whether the token can be extended to a longer match by checking if the current configuration intersects with the oracle entry at the next position ($S \cap \text{oracle}[\text{pos} + 1] = \emptyset$ means the token cannot be extended). Both algorithms eliminate backtracking through preprocessing and achieve linear time complexity with reduced memory requirements. Their work extends beyond traditional compiler lexical analysis to data extraction tasks (parsing JSON, CSV, and other semi-structured formats), demonstrating that optimized versions of their algorithms perform competitively with existing tokenizers while providing stronger theoretical guarantees on time and space complexity. This research was published in *Proceedings of the ACM on Programming Languages* (OOPSLA 2024) and represents significant progress toward making tokenization both theoretically sound and practically efficient.

[5]An **AST (Abstract Syntax Tree)** is a tree representation of the syntactic structure of source code where each node represents a construct in the language. Unlike concrete syntax trees (parse trees), ASTs abstract away syntactic details like parentheses and semicolons, retaining only semantically meaningful structure. For example, the expression `2 + 3 * 4` produces an AST with a multiplication node (children: 3 and 4) as the right child of an addition node (left child: 2), directly encoding operator precedence.

[6]A **context-free grammar (CFG)** is a formal specification of a language's syntax using production rules of the form $A \to \alpha$, where A is a nonterminal symbol and α is a sequence of terminals and nonterminals. For example, a simple expression grammar might include rules like $E \to E + T \mid T$ and $T \to T * F \mid F$, where E (expression), T (term), and F (factor) are nonterminals, and $+$, $*$ are terminals. CFGs are more powerful than regular expressions (used in lexical analysis) because they can express nested structures like balanced parentheses, but less powerful than context-sensitive grammars. The name "context-free" reflects that the left-hand side is always a single nonterminal, meaning the production can be applied regardless of surrounding context.

[7]**Top-down parsing** constructs the parse tree from the root downward, attempting to match the input by predicting which production rules to apply. **Recursive descent** is a simple top-down technique where each nonterminal has a corresponding parsing function that calls other functions recursively. **LL(k) grammars** (Left-to-right scan, Leftmost derivation, k tokens lookahead) are grammars parseable by top-down methods with k tokens of lookahead. For example, LL(1) parsers make decisions using only the current token. The production rule $S \to aB \mid aC$ is LL(1) if the FIRST sets of aB and aC are disjoint (i.e., we can decide which production to use by looking at the next token). Top-down parsers are intuitive and easy to implement by hand but cannot handle left-recursive grammars (rules like $E \to E + T$) without transformation.

[8]**Bottom-up parsing** builds the parse tree from the leaves upward, attempting to reduce sequences of terminals and nonterminals back to the start symbol. **LR(k) parsers** (Left-to-right scan, Rightmost derivation in reverse, k tokens lookahead) are the most powerful deterministic bottom-up parsers, capable of parsing nearly all deterministic CFGs. **SLR (Simple LR)** parsers use simplified lookahead computation and are easier to construct but less powerful. **LALR(1) (Lookahead LR)** parsers strike a balance: they are more powerful than SLR but use the same amount of memory by merging parser states with identical cores. Most programming languages (C, Java, JavaScript) use LALR(1) grammars because they can express natural language syntax while keeping parser tables manageable. Bottom-up parsers use a *shift-reduce* strategy: *shift* pushes the next token onto the stack, *reduce* replaces the top stack symbols matching the right-hand side of a production with the left-hand side nonterminal. For example, parsing `2 + 3` with grammar $E \to E + E \mid \text{num}$: shift 2, reduce to E, shift $+$, shift 3, reduce to E, reduce $E + E$ to E.

[9]**Parser generators** are tools that automatically generate parser code from grammar specifications. **YACC (Yet Another Compiler-Compiler)** and its GNU implementation **Bison** are classic LALR(1) parser generators that take grammar rules annotated with semantic actions (C code) and produce a C parser. Modern alternatives include ANTLR (LL(*) parser generator supporting multiple target languages), Tree-sitter (incremental parsing for text editors), and PEG (Parsing Expression Grammar) parsers. Parser generators eliminate the tedious and error-prone manual construction of parsing tables and allow rapid prototyping of language syntax.

[10]An **ambiguous grammar** is a CFG that can produce more than one parse tree (or derivation) for the same input string. The classic example is the expression grammar $E \to E + E \mid E * E \mid \text{num}$, which can parse `2 + 3 * 4` in multiple ways (either $(2 + 3) * 4$ or $2 + (3 * 4)$). Programming languages typically require unambiguous parsing, so parsers resolve ambiguity through **precedence** (operator priority: multiplication binds tighter than addition) and **associativity** (left-to-right: `a - b - c` means `(a - b) - c`, not `a - (b - c)`). Parser generators

strategies[11] that enable compilers to report multiple syntax errors in a single pass.

3. **Semantic Analysis:** The AST is traversed to enforce language-specific constraints not expressible in context-free grammars. Aho et al. [2] formalize this phase using *attribute grammars* and *syntax-directed translation schemes*. Cooper and Torczon [12] identify three critical tasks: (a) *type checking*—verifying that operations are applied to compatible types, (b) *scope resolution*—binding identifiers to their declarations via *symbol tables*, and (c) *semantic error detection*—identifying undefined variables, type mismatches, and uninitialized memory access. Appel [3] demonstrates symbol table construction using hash tables and discusses how static single assignment (SSA) form simplifies semantic analysis by ensuring each variable is assigned exactly once.

4. **Code Generation and Optimization:** The annotated AST is translated into target machine code through intermediate representations. Aho et al. [2] describe this as a multi-phase process involving *intermediate code generation*, *code optimization*, and *target code generation*. Cooper and Torczon [12] emphasize modern optimization techniques including *dataflow analysis*, *register allocation via graph coloring*, and *instruction scheduling*. Appel [3] provides detailed algorithms for *instruction selection via tree matching* and discusses runtime system integration (stack frames, calling conventions, garbage collection).

The compilation phases differ in emphasis across authoritative texts: the Dragon Book prioritizes formal foundations (automata theory, formal grammars, attribute grammars), Appel's work emphasizes practical implementation with complete working examples in ML/Java/C, while Cooper and Torczon focus on engineering pragmatics and modern optimization algorithms. All three recognize that real-world compilers interleave these phases and employ multiple passes to achieve correctness and performance.

The output is a standalone executable binary containing machine instructions that the CPU can execute directly. Compiled languages (C, C++, Rust, Go) typically offer superior runtime performance because the translation overhead occurs only once, at compile time. However, compiled binaries are architecture-specific and must be recompiled for different target platforms.

1.2.2 Interpreter

An interpreter translates and executes source code *line-by-line or statement-by-statement at runtime*, without producing a separate executable file. The interpreter reads source code, parses it into an internal representation, and immediately executes the corresponding operations. Interpreted languages (Python, Ruby, JavaScript in early implementations) provide greater flexibility and portability (source code runs on any platform with an interpreter), but suffer performance penalties due to repeated parsing and interpretation overhead during execution.

allow declaring these properties (e.g., `%left '+'` in YACC), enabling parsers to handle naturally ambiguous grammars by imposing disambiguation rules. The "dangling else" problem in if-statements (`if (a) if (b) x; else y;`) is another classic ambiguity resolved by associating `else` with the nearest unmatched `if`.

[11] **Error recovery strategies** enable parsers to continue parsing after encountering syntax errors, reporting multiple errors in a single compilation pass rather than aborting at the first error. Common techniques include: **panic mode** (skip tokens until a synchronizing token like semicolon or closing brace is found), **phrase-level recovery** (perform local corrections like inserting or deleting tokens), **error productions** (explicitly add common error patterns to the grammar), and **global correction** (find the minimal sequence of insertions/deletions to make the input parseable, computationally expensive). Modern parsers (GCC, Clang, TypeScript compiler) employ sophisticated heuristics to provide helpful error messages, pointing developers to the likely location and nature of syntax errors. Incremental parsers used in IDEs must handle incomplete or syntactically invalid code gracefully to provide real-time feedback.

1.2.3 Assembler

An assembler translates assembly language source code into machine code (binary object files). Since assembly instructions have a nearly one-to-one correspondence with machine opcodes, assemblers perform a relatively straightforward translation. Assemblers are essential in the compilation pipeline, as many compilers generate assembly as an intermediate step before producing final machine code.

The use of assembly as a **low-level intermediate representation (low-level IR)** serves several important purposes in traditional compiler design. This approach is commonly referred to as **assembly-based code generation** or using an **assembly backend**. First, it provides **separation of concerns**: the compiler focuses on high-level optimizations and instruction selection during the **code generation** phase, while the assembler handles architecture-specific details like instruction encoding, relocation, and symbol resolution during the final **code emission** stage. Second, assembly output is **human-readable**, enabling developers to inspect generated code for debugging and performance optimization. Third, it offers **modularity and flexibility**: different compilers can share the same assembler, and hand-written assembly modules can be integrated seamlessly into compiled programs.

Many production compilers adopt this **multi-pass compilation** strategy with assembly as a low-level IR. The **GNU Compiler Collection (GCC)** generates assembly code (typically with a `.s` extension) and then invokes the GNU Assembler (`gas`, part of GNU Binutils) to produce object files. The **Rust compiler (`rustc`)** uses LLVM as its backend and can emit assembly via the `-emit asm` flag. Most traditional **C/C++ compilers** historically followed this two-stage pipeline, producing assembly before final object code.

However, modern compiler infrastructures increasingly support **direct machine code generation** without assembly intermediates. **LLVM/Clang** implements an *integrated assembler* that enables *direct object emission*: the compiler backend generates object files directly from LLVM IR, bypassing the textual assembly stage entirely. This approach eliminates the overhead of serializing to assembly syntax and parsing it back, resulting in faster compilation. By default, Clang uses LLVM's integrated assembler on all supported targets, though it can also invoke external assemblers when needed for compatibility or debugging purposes.

1.2.4 Linker

A linker combines multiple object files (compiled machine code modules) and resolves external references (function calls, global variables) to produce a final executable or shared library. The linker performs address relocation, symbol resolution, and merges code and data sections. Modern compilation systems typically invoke the linker automatically as the final stage of the build process.

In practice, modern language implementations often employ **hybrid approaches**: Just-In-Time (JIT) compilation compiles frequently executed code paths at runtime (Java JVM, JavaScript V8 engine), combining the portability of interpretation with the performance of compilation. Languages like TypeScript are transpiled to JavaScript, which is then JIT-compiled by the browser's JavaScript engine.

Traditional compilers are monolithic: each language needs its own complete toolchain targeting every architecture. If you have M languages and N architectures, you need $M \times N$ implementations. This becomes unsustainable as both language diversity and hardware complexity grow.

LLVM solves this by introducing an **IR**—a stable, language-independent, architecture-independent format that sits between high-level source code and low-level machine code. This concept is philosophically similar to Java's bytecode and the JVM, which we discussed earlier as a successful example of "write once, run anywhere." However, while the JVM operates as a runtime virtual machine executing bytecode, LLVM IR serves as a compile-time intermediate

representation that gets optimized and compiled to native machine code for maximum performance. In essence, LLVM provides the portability benefits of the JVM approach while delivering the execution speed of native compilation.

Classical compiler theory suggests that M languages targeting N architectures require $M \times N$ distinct back-ends. In practice, LLVM collapses this to M front-ends plus lightweight target descriptions, because the bulk of back-end logic (instruction selection, scheduling, register allocation, code emission) is shared within LLVM itself. You parameterize the backend with a data layout specification, calling convention, and a modest set of target-specific hooks rather than re-implementing a complete code generator for each architecture. This reduces compilation process complexity from $M \times N$ to $M + N$.

1.3 The LLVM IR Project

LLVM began as an open-source compiler infrastructure project at the University of Illinois in 2000, originally designed to provide a modern, SSA-based compilation technique for arbitrary static and dynamic programming languages.

The design and core concepts developed in the LLVM research project have proven highly advantageous for language implementation and compilation due to its unified ISA and the **conceptual similarity** between LLVM IR and real assembly languages. Consequently, the project has evolved into a comprehensive ecosystem of distinct subprojects providing reusable libraries and ISA specifications.

1.3.1 The Compilation Pipeline

Traditional compilation flows from source code → AST → assembly → machine code. Given the architecture diversity discussed in the previous section, this creates the $M \times N$ problem: M languages each needing N architecture-specific backends.

To address this challenge, LLVM introduced a **unified abstract low-level language** with a common ISA that is structurally analogous to native assembly languages. The compilation process (translation to machine code for each architecture) is then executed by the LLVM compiler infrastructure, which acts as a virtual machine for this abstract representation.

LLVM IR is therefore a low-level intermediate language that resembles assembly but with several critical differences. First, it is ***strongly typed***—every value has an explicit type (`i32`, `float`, `ptr`, etc.), enabling compile-time type checking and optimization opportunities. Second, it is ***platform-independent***, meaning the same IR code works for x86, ARM, GPU, and WASM targets without modification. Third, it enforces ***SSA form***, where every variable is assigned exactly once, enabling aggressive optimizations by simplifying data flow analysis. Fourth, it provides ***infinite registers***—virtual registers prefixed with `%` that the backend maps to physical hardware registers during code generation. Finally, it uses ***explicit control flow*** through structured basic blocks and explicit branch instructions instead of unstructured goto statements.

Example LLVM IR function:

```
define i32 @add(i32 %a, i32 %b) {
entry:
  %result = add i32 %a, %b
  ret i32 %result
}
```

This representation is simultaneously human-readable (unlike binary machine code), optimizable (unlike high-level source code), and portable (unlike assembly language).

1.3.2 Optimization Pipeline

As mentioned above, the LLVM infrastructure compiles (translates to machine code) the intermediate representation of instructions for each specific architecture. However, thanks to the use of the SSA concept, the resulting instruction graph structure can be optimized—meaning code with better performance characteristics can be generated for the target architecture. Unfortunately, there is no simple universal definition of *optimal code* with respect to performance; optimality is determined by the specific requirements and design goals of each application.

LLVM supports six optimization levels, each applying progressively more aggressive transformations:

`-O0` **(No optimization)** This level performs **no optimizations whatsoever**. The compiler generates machine code that directly corresponds to the IR instructions, making the relationship between source code and assembly trivial to understand. This is essential for debugging, as it preserves variable locations, call stack integrity, and stepping behavior. Compilation is fastest at this level. Use `-O0` during active development and debugging sessions.

`-O1` **(Basic optimization)** Applies **fundamental optimizations** that improve performance with minimal compilation time overhead. These include **constant folding** (evaluating constant expressions at compile time, e.g., `x = 3 * 4` becomes `x = 12`), **dead code elimination** (removing unreachable code and unused variables), **simple inlining**[12] (inlining trivial functions such as getters, setters, and small wrappers), and **expression simplification** (reducing algebraic expressions, e.g., `x * 1` becomes `x`). Use `-O1` when you want some performance improvement while maintaining reasonable compile times and debuggability.

`-O2` **(Moderate optimization)** The **recommended level for production code**. Applies all `-O1` optimizations plus **loop optimizations** (loop unrolling, loop invariant code motion, strength reduction), **vectorization** (automatically converting scalar operations to SIMD instructions such as SSE and AVX on x86 or NEON on ARM), **function inlining** (aggressive inlining of hot functions based on profitability heuristics), **CSE** (reusing results of repeated calculations), **register allocation optimization** (better use of physical registers, reducing memory spills), and **instruction scheduling** (reordering instructions to minimize CPU pipeline stalls). This level provides an excellent balance between compilation time and runtime performance. Most production software uses `-O2`.

`-O3` **(Aggressive optimization)** Applies all `-O2` optimizations plus **aggressive transformations that may increase code size**, including **aggressive loop transformations** (loop interchange, loop fusion, polyhedral optimization), **aggressive inlining** (inlining larger functions, potentially duplicating code across call sites), **IPO** (analyzing and optimizing across function boundaries), **tail call optimization** (converting recursive calls to iterative loops where possible), and **speculative execution** (duplicating code paths to enable better scheduling). Use `-O3` for compute-intensive workloads (numerical computing, machine learning, GPU kernels) where performance is critical and binary size is not a concern. Note that `-O3` can occasionally introduce numerical instability in floating-point code due to aggressive reassociation.

`-Os` **(Optimize for size)** Applies **optimizations that reduce binary size** while maintaining reasonable performance. This level enables most `-O2` optimizations except those that

[12] In compiler terminology, *inlining* refers to replacing a function call with the actual body of the function at the call site, eliminating call overhead and enabling further optimizations. This differs from the CSS concept of "inline" (inline styles or inline elements). For example, if `add(a, b)` calls a function that returns `a + b`, inlining replaces the call with the direct computation `a + b`, removing the function call entirely.

increase code size, disables aggressive inlining and loop unrolling, and prefers smaller instruction encodings when semantically equivalent. Use `-Os` for embedded systems with limited flash/ROM, WASM modules (smaller downloads), or when deploying to size-constrained environments (Docker containers, mobile apps).

`-Oz` **(Aggressively optimize for size)** Provides even more **aggressive size reduction than** `-Os` by disabling vectorization (SIMD instructions are larger than scalar equivalents), applying aggressive function outlining (extracting common code sequences into shared functions), and potentially sacrificing performance for size reduction. Use `-Oz` when binary size is the absolute priority, such as bootloaders, firmware, or extremely resource-constrained IoT devices.

In sections 2.3 and 6.6, we comprehensively demonstrate how to apply each of these optimization strategies to CPU and GPU code generation workflows.

1.3.3 Portability

One of LLVM IR's most significant advantages is **true cross-platform portability**. Unlike traditional assembly languages where `x86` code, ARM code, and RISC-V code are fundamentally incompatible, LLVM IR provides a **write once, compile anywhere** paradigm.

The same IR code can be compiled to native machine code for x86-64, ARM, RISC-V, MIPS, PowerPC, GPU architectures (NVIDIA, AMD, Intel), and even WASM targets *without modification*. The LLVM backend automatically handles all architecture-specific details—instruction selection, register allocation, calling conventions, and alignment requirements—transparently during code generation.

This portability eliminates the traditional problem where developers must maintain separate assembly codebases for each target platform. Instead of writing and debugging architecture-specific implementations, you write IR once, and the LLVM infrastructure ensures correct compilation for all supported targets. This dramatically reduces maintenance burden and enables seamless cross-platform deployment.

For performance-critical sections traditionally written in inline assembly (cryptography routines, vectorized numeric kernels, system-level operations), LLVM IR offers comparable performance while maintaining portability. The IR abstracts away hardware differences while preserving low-level control over memory layout, instruction sequencing, and optimization hints.

As programming technologies evolve, the LLVM project continuously adapts by introducing **specialized IR dialects** and extensions that support emerging compilation requirements. For instance, **NVVM IR** (NVIDIA Virtual Machine) extends standard LLVM IR with GPU-specific intrinsics for thread indexing, shared memory management, and synchronization primitives required for CUDA compilation. Similarly, **AMDGPU IR** provides AMD-specific extensions, and **SPIR-V** serves as a portable intermediate representation for OpenCL and Vulkan compute shaders. These dialects maintain full compatibility with the core LLVM infrastructure while enabling platform-specific optimizations and features, ensuring that LLVM remains current with evolving hardware capabilities and programming paradigms without sacrificing its foundational portability guarantees.

1.4 Core Concepts and LLVM Syntax

This section introduces the fundamental concepts and syntax elements that form the foundation of LLVM IR programming. Some concepts are unique to LLVM, while others derive from established compiler theory; we explain the rules and conventions for their usage within LLVM IR.

1.4.1 Static Single Assignment (SSA) Form

SSA is a property where each variable is assigned exactly once and every use refers to a specific definition. This constraint makes data flow explicit and enables powerful compiler optimizations by simplifying the analysis of how values propagate through the program.

Non-SSA code:

```
x = 1
x = x + 2  // x is assigned twice (not SSA)
```

SSA code:

```
%x1 = 1
%x2 = add i32 %x1, 2  // Each value has unique name
```

The benefits of SSA form include:

- **Constant propagation**: The value of `%x2` is provably 3, enabling compile-time evaluation
- **Dead code elimination**: Unused definitions can be safely removed without complex liveness analysis
- **Register allocation**: Clear lifetime of each value simplifies mapping virtual registers to physical registers
- **Simplified dependency tracking**: Each definition has a unique name, making def-use chains trivial to construct

1.4.2 Modules and Target Specification

LLVM IR is organized hierarchically, with the **module** serving as the top-level container. A module corresponds to a single compilation unit and contains functions, global variables, type definitions, and metadata.

By convention, LLVM IR files begin with comments identifying the module and source:

```
; ModuleID = 'my_module.ll'
; source_filename = "my_source.c"
```

These comments are automatically generated by compilers and help identify the module's origin during debugging. The `@euriklis/llvm-ir` library provides dedicated methods to generate this metadata programmatically, ensuring proper module identification in generated code.

Every module must specify its target architecture through two declarations:

```
target triple = "x86_64-unknown-linux-gnu"
target datalayout = "e-m:e-p270:32:32-i64:64-f80:128-n8:16:32:64-S128"
```

The **target triple** follows the format `architecture-vendor-os-environment` and identifies the platform:

- `x86_64-unknown-linux-gnu` — Linux on x86-64
- `x86_64-apple-darwin` — macOS on Intel
- `aarch64-unknown-linux-gnu` — Linux on ARM (AWS Graviton, Raspberry Pi)
- `aarch64-apple-darwin` — macOS on Apple Silicon
- `i686-unknown-linux-gnu` — Linux on x86 32-bit
- `riscv64-unknown-linux-gnu` — Linux on RISC-V 64-bit
- `powerpc64le-unknown-linux-gnu` — Linux on PowerPC 64-bit little-endian
- `nvptx64-nvidia-cuda` — NVIDIA GPUs
- `amdgcn-amd-amdhsa` — AMD GPUs (ROCm)
- `wasm32-unknown-unknown` — WASM
- `spirv64-unknown-unknown` — SPIR-V for Vulkan/OpenCL

To view the complete list of supported target triples on your system, run `llc -version`, which displays all registered targets. For detailed information about specific targets, use `llc -march=<arch> -mattr=help` to see available CPU features and subtargets.

Example (LLVM 18.1.3):

```
$ llc --version
Ubuntu LLVM version 18.1.3
  Optimized build.
  Default target: x86_64-pc-linux-gnu
  Host CPU: znver2

  Registered Targets:
    aarch64     - AArch64 (little endian)
    aarch64_32  - AArch64 (little endian ILP32)
    aarch64_be  - AArch64 (big endian)
    amdgcn      - AMD GCN GPUs
    arm         - ARM
    arm64       - ARM64 (little endian)
    arm64_32    - ARM64 (little endian ILP32)
    armeb       - ARM (big endian)
    avr         - Atmel AVR Microcontroller
    bpf         - BPF (host endian)
    bpfeb       - BPF (big endian)
    bpfel       - BPF (little endian)
    hexagon     - Hexagon
    lanai       - Lanai
    loongarch32 - 32-bit LoongArch
    loongarch64 - 64-bit LoongArch
    m68k        - Motorola 68000 family
    mips        - MIPS (32-bit big endian)
    mips64      - MIPS (64-bit big endian)
    mips64el    - MIPS (64-bit little endian)
    mipsel      - MIPS (32-bit little endian)
    msp430      - MSP430 [experimental]
    nvptx       - NVIDIA PTX 32-bit
    nvptx64     - NVIDIA PTX 64-bit
    ppc32       - PowerPC 32
    ppc32le     - PowerPC 32 LE
    ppc64       - PowerPC 64
    ppc64le     - PowerPC 64 LE
    r600        - AMD GPUs HD2XXX-HD6XXX
    riscv32     - 32-bit RISC-V
    riscv64     - 64-bit RISC-V
    sparc       - Sparc
    sparcel     - Sparc LE
    sparcv9     - Sparc V9
    systemz     - SystemZ
    thumb       - Thumb
    thumbeb     - Thumb (big endian)
    ve          - VE
    wasm32      - WebAssembly 32-bit
    wasm64      - WebAssembly 64-bit
    x86         - 32-bit X86: Pentium-Pro and above
    x86-64      - 64-bit X86: EM64T and AMD64
    xcore       - XCore
    xtensa      - Xtensa 32
```

Example of target-specific features (NVIDIA PTX):

```
$ llc -march=nvptx64 -mattr=help
Available CPUs for this target:

  sm_20  - Select the sm_20 processor.
  sm_21  - Select the sm_21 processor.
  sm_30  - Select the sm_30 processor.
  sm_32  - Select the sm_32 processor.
  sm_35  - Select the sm_35 processor.
  sm_37  - Select the sm_37 processor.
  sm_50  - Select the sm_50 processor.
  sm_52  - Select the sm_52 processor.
  sm_53  - Select the sm_53 processor.
  sm_60  - Select the sm_60 processor.
  sm_61  - Select the sm_61 processor.
  sm_62  - Select the sm_62 processor.
  sm_70  - Select the sm_70 processor.
  sm_72  - Select the sm_72 processor.
  sm_75  - Select the sm_75 processor.
  sm_80  - Select the sm_80 processor.
  sm_86  - Select the sm_86 processor.
  sm_87  - Select the sm_87 processor.
  sm_89  - Select the sm_89 processor.
  sm_90  - Select the sm_90 processor.
  sm_90a - Select the sm_90a processor.

Available features for this target:

  ptx32  - Use PTX version 32.
  ptx40  - Use PTX version 40.
  ptx41  - Use PTX version 41.
  ptx42  - Use PTX version 42.
  ptx43  - Use PTX version 43.
  ptx50  - Use PTX version 50.
  ptx60  - Use PTX version 60.
  ptx61  - Use PTX version 61.
  ptx63  - Use PTX version 63.
  ptx64  - Use PTX version 64.
  ptx65  - Use PTX version 65.
  ptx70  - Use PTX version 70.
  ptx71  - Use PTX version 71.
  ptx72  - Use PTX version 72.
  ptx73  - Use PTX version 73.
  ptx74  - Use PTX version 74.
  ptx75  - Use PTX version 75.
  ptx76  - Use PTX version 76.
  ptx77  - Use PTX version 77.
  ptx78  - Use PTX version 78.
  ptx80  - Use PTX version 80.
  ptx81  - Use PTX version 81.
  ptx82  - Use PTX version 82.
  ptx83  - Use PTX version 83.
  sm_20  - Target SM 20.
  sm_21  - Target SM 21.
  sm_30  - Target SM 30.
  sm_32  - Target SM 32.
  sm_35  - Target SM 35.
  sm_37  - Target SM 37.
```

```
  sm_50   - Target SM 50.
  sm_52   - Target SM 52.
  sm_53   - Target SM 53.
  sm_60   - Target SM 60.
  sm_61   - Target SM 61.
  sm_62   - Target SM 62.
  sm_70   - Target SM 70.
  sm_72   - Target SM 72.
  sm_75   - Target SM 75.
  sm_80   - Target SM 80.
  sm_86   - Target SM 86.
  sm_87   - Target SM 87.
  sm_89   - Target SM 89.
  sm_90   - Target SM 90.
  sm_90a  - Target SM 90a.

Use +feature to enable a feature, or -feature to disable it.
For example, llc -mcpu=mycpu -mattr=+feature1,-feature2
```

The **data layout string** is a compact specification that encodes essential low-level architectural properties required for correct code generation [21]. This string informs the LLVM optimizer and backend about target-specific characteristics such as endianness, pointer sizes, type alignments, and memory layout conventions [9]. Without accurate data layout information, the compiler cannot generate correct memory access patterns, properly align data structures, or make sound optimization decisions that depend on type sizes and alignment constraints [20].

Understanding the data layout string reveals how the same IR code can behave differently across platforms. Consider a simple operation like storing a 32-bit integer: the compiler must know **endianness** (byte ordering in memory), **alignment** (valid memory addresses for types), and **pointer size** (32-bit vs. 64-bit addressing). These architectural differences are abstracted away in high-level languages but become critical when generating machine code. When LLVM compiles your IR, it uses the data layout to:

1. **Generate correct memory instructions**: Determine how to load/store multi-byte values (e.g., reading a 64-bit integer requires knowing if bytes are stored least-significant-first or most-significant-first)
2. **Calculate struct padding**: Insert padding bytes between struct fields to meet alignment requirements, ensuring efficient memory access
3. **Optimize memory access patterns**: Use SIMD instructions when data is properly aligned, or fall back to slower unaligned access when necessary
4. **Implement pointer arithmetic**: Correctly compute memory offsets for array indexing and structure field access based on target-specific type sizes

Think of the data layout as a contract between your IR code and the target hardware—it ensures the compiled binary correctly interacts with physical memory and CPU registers.

Data Layout String Format

The data layout specification consists of multiple components separated by the minus sign (-), where each component begins with a letter identifying the specification type, followed by parameters [21]. A typical x86-64 data layout string is:

```
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80
   :128-n8:16:32:64-S128"
```

Component Breakdown

Endianness (`e` or `E`): Endianness defines the order in which bytes of a multi-byte value are stored in memory. When you store a number larger than one byte (like a 32-bit integer), the CPU must decide: should the first byte in memory be the most significant or least significant? Different CPU architectures make different choices.

The first character in the data layout specifies byte ordering [21]:

- `e` — Little-endian: least significant byte at lowest memory address (Intel/AMD x86, ARM in little-endian mode)
- `E` — Big-endian: most significant byte at lowest memory address (legacy PowerPC, SPARC, some network protocols)

Example: Consider the 32-bit integer `0x12345678`. This number occupies 4 bytes in memory:

```
Address:  0x1000  0x1001  0x1002  0x1003
Little:     0x78    0x56    0x34    0x12   (least significant first)
Big:        0x12    0x34    0x56    0x78   (most significant first)
```

Why it matters: When reading data from disk, network, or interfacing with hardware, the endianness must match. Reading a little-endian value as big-endian would interpret `0x12345678` as `0x78563412`—completely wrong!

Name Mangling (`m:<style>`): Specifies how symbol names are mangled for the linker [21]:

- `m:e` — ELF mangling (Linux, BSD, most Unix systems)
- `m:o` — Mach-O mangling (macOS, iOS)
- `m:m` — Mips mangling
- `m:w` — Windows COFF mangling
- `m:x` — Windows COFF x86 mangling (different from `m:w` for compatibility)

Symbol mangling affects how function and global variable names are encoded in object files, determining linker compatibility and calling convention details.

Pointer Layout (`p[n]:<size>:<abi>:<pref>`): Specifies pointer characteristics for address space `n` [21]. If `n` is omitted, address space 0 (default) is assumed:

- `<size>` — Pointer size in bits
- `<abi>` — ABI-required alignment in bits (minimum guaranteed by calling conventions)
- `<pref>` — Preferred alignment in bits for performance (optional; if omitted, equals `<abi>`)

Examples from the x86-64 data layout:

- `p270:32:32` — Address space 270: 32-bit pointers with 32-bit alignment (used for `__ptr32` Microsoft extension)
- `p271:32:32` — Address space 271: 32-bit zero-extended pointers (used for `__uptr`)
- `p272:64:64` — Address space 272: 64-bit pointers with 64-bit alignment (used for `__ptr64`)
- Default (`p:64:64:64`) — Address space 0: 64-bit pointers with 64-bit alignment (standard pointers)

Multiple address spaces enable heterogeneous programming where different pointer types have different sizes (e.g., GPU programming with separate host and device address spaces) [9].

Integer Alignment (`i<size>:<abi>:<pref>`): Alignment specifies which memory addresses are valid for storing a value. A value is **aligned** if its memory address is divisible by its size. For example, a 32-bit (4-byte) integer should be stored at addresses divisible by 4 (like 0x1000, 0x1004, 0x1008), not at arbitrary addresses like 0x1001.

The alignment specification format is `i<size>:<abi>:<pref>` [21]:

```
i64:64      ; i64 has 64-bit (8-byte) ABI and preferred alignment
i32:32      ; i32 has 32-bit (4-byte) alignment
i8:8        ; i8 has 8-bit (1-byte) alignment
```

Example: Storing an `i32` (4-byte integer) in memory:

```
Valid addresses (aligned):   0x1000, 0x1004, 0x1008, 0x100C
Invalid addresses (misaligned): 0x1001, 0x1002, 0x1003, 0x1005
```

Why alignment matters:

- **Performance**: CPUs read memory in aligned chunks (cache lines). An aligned 64-bit read takes one memory transaction; a misaligned read may require two transactions, doubling access time
- **Correctness**: Some CPUs (ARM, SPARC) raise hardware exceptions or crashes on misaligned access
- **Atomic operations**: CPU atomic instructions (compare-and-swap, etc.) require natural alignment; misaligned atomics may silently produce incorrect results

Floating-Point Alignment (`f<size>:<abi>:<pref>`): Specifies alignment for floating-point types [21]:

```
f80:128     ; x86 80-bit extended precision (stored in 128-bit slots)
f64:64      ; double (64-bit float)
f32:32      ; float (32-bit float)
```

The `f80:128` entry indicates that while x86 extended precision floats are 80 bits, they are stored in 128-bit-aligned memory slots for efficiency.

Vector Alignment (`v<size>:<abi>:<pref>`): Specifies alignment for vector types used in SIMD operations [21]:

```
v128:128    ; 128-bit vectors (SSE on x86, NEON on ARM)
v256:256    ; 256-bit vectors (AVX)
v512:512    ; 512-bit vectors (AVX-512)
```

Proper vector alignment is mandatory for SIMD instruction efficiency. Misaligned vector loads/stores can cause significant performance penalties or instruction faults.

Aggregate Alignment (`a:<abi>:<pref>`): Specifies alignment for aggregate types (structs, arrays) [21]:

```
a:0:64      ; Aggregates have 0-bit ABI alignment, 64-bit preferred
   alignment
```

This allows the compiler to optimize struct layout while maintaining ABI compatibility.

Native Integer Widths (`n<size1>:<size2>:...`): Specifies integer widths that the target CPU can efficiently process in a single instruction [21]:

```
n8:16:32:64 ; x86-64 efficiently supports 8, 16, 32, and 64-bit
   integers
n32:64      ; PowerPC 64 efficiently supports 32 and 64-bit integers
```

The optimizer uses this information for type legalization decisions. Operations on ***non-native widths*** (e.g., `i7`, `i128` on x86-64) are decomposed into multiple native-width operations.

Stack Alignment (`S<size>`): Specifies the natural stack alignment in bits [21]:

```
S128        ; Stack maintains 128-bit (16-byte) alignment
```

Stack alignment affects:

- Function prologues/epilogues (stack frame setup)
- SIMD variable allocation (local vectors must be aligned)
- Calling conventions (some ABIs require specific stack alignment)

On x86-64, 16-byte stack alignment enables efficient SSE operations on stack-allocated vectors [9].

The data layout string is essential for three fundamental reasons [20]: **(1) Correctness**—wrong alignment can cause crashes on strict-alignment architectures or incorrect atomic operations; **(2) Performance**—proper alignment enables efficient memory access, SIMD vectorization, and cache utilization; and **(3) ABI compliance**—data layout must match the platform ABI to ensure interoperability with system libraries and other compiled code. Modern LLVM automatically sets the data layout when you specify the target triple, but understanding the format is crucial for cross-compilation, GPU programming, and custom target development [9].

1.4.3 Global and Local Variables

LLVM distinguishes between global variables (module-level storage) and local variables (function-level virtual registers). In LLVM terminology, these constructs are called **identifiers**. Global identifiers (functions, global variables) begin with the `@` character, while local identifiers (virtual registers, local names) begin with the `%` character. This prefix-based naming scheme serves a crucial design purpose: it prevents conflicts with reserved words (such as opcodes like `add`, `mul`, or type names like `i32`, `void`). Since reserved words in LLVM never start with `@` or `%`, identifiers can never clash with them. This design provides flexibility — LLVM can expand its set of reserved words in future versions without breaking existing code, as no valid identifier can conflict with a reserved word.

Identifiers come in two forms: **named** (e.g., `%foo`, `@DivisionByZero`) and **unnamed** (e.g., `%12`, `@2`). Named identifiers follow the regular expression pattern `[-a-zA-Z$._][-a-zA-Z$._0-9]*`. Unnamed identifiers allow compilers to quickly generate temporary values without symbol table lookups.

Global variables are declared outside functions and persist for the program's lifetime:

```
@global_counter = global i32 0
@constant_pi = constant float 3.14159
@external_array = external global [100 x i32]
```

Global variables use the `@` prefix and have linkage specifiers: ***global*** (mutable global variable), ***constant*** (immutable global constant), ***external*** (declared elsewhere, no initializer), and ***internal*** (private to the module). See section 1.4.5 for more details.

Local variables (virtual registers) use the `%` prefix and exist only within a function:

```
define i32 @compute(i32 %x) {
entry:
  %temp1 = add i32 %x, 5
  %temp2 = mul i32 %temp1, 2
  ret i32 %temp2
}
```

In SSA form, each local variable is assigned exactly once. For mutable variables (stack allocations), LLVM uses memory operations:

```
define void @mutable_example() {
entry:
  %ptr = alloca i32               ; Allocate stack space
  store i32 42, ptr %ptr          ; Write to memory
  %val = load i32, ptr %ptr       ; Read from memory
  ret void
}
```

SSA Values vs. Memory Locations

Understanding the distinction between SSA values and memory locations is crucial:

SSA values (virtual registers with `%` prefix) can only be assigned **once**:

```
%x = add i32 1, 2       ; %x = 3
%x = mul i32 4, 5       ; ERROR: Cannot reassign %x (violates SSA)
```

Each new value requires a new variable name:

```
%x = add i32 1, 2       ; %x = 3
%y = mul i32 %x, 5      ; %y = 15 (new variable name)
%z = add i32 %y, 1      ; %z = 16 (new variable name)
```

Memory locations (globals and `alloca`) can be stored to **multiple times**:

```
@global_var = global i32 0

define void @modify_memory() {
  %local_ptr = alloca i32

  ; Multiple stores to global memory
  store i32 10, ptr @global_var
  store i32 20, ptr @global_var    ; OK: Overwrites previous value
  store i32 30, ptr @global_var    ; OK: Overwrites again

  ; Multiple stores to local memory
  store i32 100, ptr %local_ptr
  store i32 200, ptr %local_ptr    ; OK: Overwrites previous value

  ret void
}
```

Key distinction: The pointer variable (`%local_ptr`) is assigned once (the `alloca` result), but the **memory location** it points to can be modified multiple times. Each `load` creates a new SSA value:

```
%ptr = alloca i32               ; %ptr assigned once (address)
store i32 10, ptr %ptr
%val1 = load i32, ptr %ptr      ; %val1 = 10 (new SSA value)
store i32 20, ptr %ptr          ; Modify memory
%val2 = load i32, ptr %ptr      ; %val2 = 20 (new SSA value)
```

This pattern allows mutable variables while maintaining SSA form: the pointer names follow SSA rules, but the memory they point to can change.

1.4.4 Functions

Functions are the primary units of executable code in LLVM IR. A function declaration consists of:

```
define <return_type> @function_name(<param_type> %param1, ...) {
  ; function body
}
```

The observant reader has probably noticed that the naming convention for functions in LLVM matches that of global variables—both use the `@` prefix. This similarity is not coincidental: **functions are global values** in LLVM IR. Like global variables, functions are:

- Declared at **module scope** (not inside other functions)

- Visible across the entire module
- Have **addresses** that can be referenced and stored
- Support **linkage types** controlling visibility and linking behavior

Functions can be treated as values and stored in function pointers:

```
define i32 @my_function(i32 %x) {
  %result = add i32 %x, 1
  ret i32 %result
}

@function_ptr = global ptr @my_function  ; Store function address

define void @call_via_pointer() {
  %fptr = load ptr, ptr @function_ptr
  %result = call i32 %fptr(i32 42)       ; Call through pointer
  ret void
}
```

In LLVM's type hierarchy, both `Function` and `GlobalVariable` inherit from `GlobalValue`, making them module-level entities with global visibility.

Linkage types control visibility and linking behavior:

- `define` — Default linkage (internal to module unless exported)
- `define external` — Exported symbol, visible to other modules
- `define internal` — Private to the module (similar to C `static`)
- `define weak` — Can be overridden by other definitions
- `declare` — External function (declaration only, no body)

Function attributes modify compilation behavior:

```
define i32 @pure_function(i32 %x) nounwind readnone {
  %result = mul i32 %x, %x
  ret i32 %result
}
```

Common attributes:

- `nounwind` — Function never throws exceptions
- `readnone` — Pure function, no memory access
- `readonly` — Function only reads memory
- `alwaysinline` — Force inlining
- `noinline` — Prevent inlining

1.4.5 Type System

LLVM IR is strongly typed. Every value has an explicit type, and type mismatches are compile-time errors. The type system includes primitive types, aggregate types, and pointer types.

Primitive types:

- **Integer types**: `i1` (boolean), `i8`, `i16`, `i32`, `i64`, `i128` (arbitrary-width integers like `i7` or `i256` are also valid)
- **Floating-point types**: `half` (16-bit), `float` (32-bit), `double` (64-bit), `fp128` (quad-precision)
- **Void type**: `void` (used only for function return types, not for values)

Aggregate types:

```
; Array: [length x element_type]
[10 x i32]                    ; Array of 10 i32 values
[3 x float]                   ; Array of 3 floats

; Structure: { type1, type2, ... }
{ i32, float, ptr }           ; Unpacked struct
<{ i8, i32 }>                ; Packed struct (no padding)

; Vector: <length x element_type>
<4 x float>                   ; SIMD vector (SSE, NEON)
<16 x i8>                     ; 128-bit packed integer vector
```

Pointer type: `ptr` (opaque pointer, introduced in LLVM 15):

```
%heap_ptr = call ptr @malloc(i64 64)
store i32 42, ptr %heap_ptr
%value = load i32, ptr %heap_ptr
```

Older LLVM versions used typed pointers (`i32*`, `float*`), but modern LLVM uses opaque `ptr` to simplify the type system and improve optimization.

Function types specify signatures:

```
; Function type: return_type (param_types)
i32 (i32, i32)                ; Takes two i32, returns i32
void (ptr, i64)               ; Takes pointer and i64, returns void
```

1.4.6 Basic Blocks and Labels

LLVM organizes function bodies into **basic blocks**—straight-line sequences of instructions with no branches except at the end. This structure is fundamental to control flow analysis and optimization.

```
define i32 @max(i32 %a, i32 %b) {
entry:
  %cmp = icmp sgt i32 %a, %b  ; Compare: a > b?
  br i1 %cmp, label %if.then, label %if.else

if.then:
  ret i32 %a

if.else:
  ret i32 %b
}
```

Each basic block has three essential properties: ***unique label***, ***sequential instructions***, and ***terminator instruction***. We examine each of these properties below.

1.4.6.1 Unique Label

Every basic block has a name (`entry`, `if.then`, `if.else`) that serves as a branch target. The label identifies the block and allows other instructions to reference it as a control flow destination. Labels must be unique within a function—if two labels with the same name appear in the same function, the compiler will reject the code with a redefinition error. Note that labels in different functions can share the same name, as label scope is local to each function. By convention, the first basic block is named `entry`.

Each instruction in a function must belong to a labeled basic block—there cannot be instructions floating outside of blocks. After a label, each instruction must appear on a new line; instructions cannot be placed on the same line as the label itself. While not syntactically required, instructions are typically indented below their block's label (usually with two spaces) to improve readability and visually distinguish labels from their instructions.

```
entry:
  %x = add i32 1, 2          ; New line, indented (typical style)
  %y = mul i32 %x, 3
  br label %next

next:
%z = sub i32 %y, 1           ; New line, not indented (valid but unusual)
ret i32 %z
```

1.4.6.2 Instructions and Opcodes

No control flow transfers occur in the middle of a basic block. Once execution enters a block at its label, all instructions execute sequentially from top to bottom until the terminator is reached. This property—single entry, single exit—is fundamental to control flow analysis and enables many compiler optimizations by simplifying data flow analysis.

LLVM instructions are the fundamental operations that manipulate values. While the exact syntax varies by instruction type, most instructions share a common structure consisting of an **optional result variable**, an **opcode** (operation name), **type information**, and **operands**. The typical form for binary operations is:

```
%result = opcode type operand1, operand2
```

For example, `%sum = add i32 %a, %b` follows this pattern. However, different instruction families have variations: comparison instructions include a *predicate* (`%cmp = icmp eq i32 %a, %b`), memory operations use explicit pointer types (`%val = load i32, ptr %ptr`), and side-effecting instructions like `store` produce no result. Despite these variations, all instructions combine an operation identifier with typed operands to perform their specific computation or effect.

The complete and authoritative documentation for all LLVM IR instructions, including detailed syntax, semantics, and behavior specifications, is provided in the **LLVM Language Reference Manual**, the official documentation maintained by the LLVM Project[13]. This comprehensive reference covers every instruction opcode, from basic arithmetic operations to advanced vector manipulations and atomic operations. The canonical instruction enumeration is maintained in the LLVM Project source code in the file `Instruction.def`[14]. Note that unlike assembly languages for specific architectures, LLVM does not provide a command-line tool to enumerate all opcodes—the Language Reference Manual serves as the definitive catalog. What follows is a concise overview of the most commonly used instruction categories with representative examples to illustrate typical usage patterns.

Arithmetic instructions:

```
%sum = add i32 %a, %b          ; Integer addition
%diff = sub i32 %a, %b         ; Integer subtraction
%prod = mul i32 %a, %b         ; Integer multiplication
```

[13] Available online at https://llvm.org/docs/LangRef.html

[14] In the LLVM source tree, this is located at `llvm/include/llvm/IR/Instruction.def`. On Ubuntu systems with LLVM installed, you can find it at `/usr/include/llvm-<version>/llvm/IR/Instruction.def` (e.g., `/usr/include/llvm-18/llvm/IR/Instruction.def`). This file contains C++ macro definitions that enumerate all instruction opcodes. You can count all instructions on Linux with: `grep "HANDLE.*INST" /usr/include/llvm-18/llvm/IR/Instruction.def | wc -l`

```
%quot = sdiv i32 %a, %b       ; Signed division
%rem = srem i32 %a, %b        ; Signed remainder

%fsum = fadd float %x, %y     ; 32-bit float addition
%fdiff = fsub float %x, %y    ; 32-bit float subtraction
%fprod = fmul float %x, %y    ; 32-bit float multiplication
%fquot = fdiv float %x, %y    ; 32-bit float division

%dsum = fadd double %x, %y    ; 64-bit double addition
%ddiff = fsub double %x, %y   ; 64-bit double subtraction
%dprod = fmul double %x, %y   ; 64-bit double multiplication
%dquot = fdiv double %x, %y   ; 64-bit double division
```

Note that floating-point opcodes (`fadd`, `fsub`, `fmul`, `fdiv`) work for all floating-point types—the type specifier (`float`, `double`, `half`, `fp128`) determines the precision.

Advanced arithmetic operations:

For transcendental functions and advanced mathematical operations, LLVM provides intrinsic functions. These are built-in functions prefixed with `llvm.` that map to efficient hardware instructions or optimized library implementations.

```
; Trigonometric functions (arguments in radians)
%s = call float @llvm.sin.f32(float %angle)
%c = call double @llvm.cos.f64(double %angle)
%t = call float @llvm.tan.f32(float %x)

; Inverse trigonometric functions
%asin = call float @llvm.asin.f32(float %x)
%acos = call double @llvm.acos.f64(double %x)
%atan = call float @llvm.atan.f32(float %x)
%atan2 = call double @llvm.atan2.f64(double %y, double %x)

; Hyperbolic functions
%sinh = call float @llvm.sinh.f32(float %x)
%cosh = call double @llvm.cosh.f64(double %x)
%tanh = call float @llvm.tanh.f32(float %x)

; Exponential and logarithmic functions
%exp = call float @llvm.exp.f32(float %x)       ; e^x
%exp2 = call double @llvm.exp2.f64(double %x)   ; 2^x
%exp10 = call float @llvm.exp10.f32(float %x)   ; 10^x
%log = call double @llvm.log.f64(double %x)     ; natural log (ln)
%log2 = call float @llvm.log2.f32(float %x)     ; log base 2
%log10 = call double @llvm.log10.f64(double %x) ; log base 10

; Power and root functions
%sqrt = call float @llvm.sqrt.f32(float %x)
%pow = call double @llvm.pow.f64(double %base, double %exp)
%powi = call float @llvm.powi.f32(float %base, i32 %exp)

; Rounding and absolute value
%floor = call float @llvm.floor.f32(float %x)
%ceil = call double @llvm.ceil.f64(double %x)
%round = call float @llvm.round.f32(float %x)
%abs = call double @llvm.fabs.f64(double %x)

; Fused multiply-add (a * b + c in one operation)
%fma = call float @llvm.fma.f32(float %a, float %b, float %c)
```

These intrinsics follow the naming convention `@llvm.<operation>.<type>`, where the type suffix indicates the floating-point precision (`f32` for `float`, `f64` for `double`, `f16` for `half`). Note that all trigonometric functions operate in radians, not degrees.

For mathematical functions not provided as LLVM intrinsics, you must declare and call external library functions from the C math library. Notably, while hyperbolic functions (`sinh`, `cosh`, `tanh`) are available as intrinsics, their inverses (`asinh`, `acosh`, `atanh`) are not:

```
; Declare external inverse hyperbolic functions
declare float @asinhf(float)
declare double @asinh(double)
declare float @acoshf(float)
declare double @acosh(double)
declare float @atanhf(float)
declare double @atanh(double)

; Call the external functions
%result_f = call float @asinhf(float %x)
%result_d = call double @asinh(double %x)
%acosh_result = call double @acosh(double %x)
%atanh_result = call float @atanhf(float %x)
```

The `declare` statement tells LLVM that these functions exist externally, but they are not implemented in your IR code. At link time, you must link against the C math library (`libm`) to resolve these function references. For example, when compiling to an executable:

```
# Compile LLVM IR to object file
llc -filetype=obj myfile.ll -o myfile.o

# Link with math library (-lm flag)
clang myfile.o -lm -o myprogram
```

Or directly with `clang`:

```
clang myfile.ll -lm -o myprogram
```

The `-lm` flag instructs the linker to include the math library where the implementations of `asinh`, `acosh`, `atanh`, and other standard math functions are found.

Bitwise operations:

```
%and_result = and i32 %a, %b  ; Bitwise AND
%or_result = or i32 %a, %b    ; Bitwise OR
%xor_result = xor i32 %a, %b  ; Bitwise XOR
%shift_left = shl i32 %a, 2   ; Shift left by 2 bits
%shift_right = lshr i32 %a, 2 ; Logical shift right
%arith_shift = ashr i32 %a, 2 ; Arithmetic shift right (sign-extend)
```

Comparison instructions:

```
; Integer comparisons (icmp)
%eq = icmp eq i32 %a, %b      ; Equal
%ne = icmp ne i32 %a, %b      ; Not equal
%gt = icmp sgt i32 %a, %b     ; Signed greater than
%lt = icmp slt i32 %a, %b     ; Signed less than
%uge = icmp uge i32 %a, %b    ; Unsigned greater or equal

; Floating-point comparisons (fcmp)
%feq = fcmp oeq float %x, %y  ; Ordered equal
%fgt = fcmp ogt float %x, %y  ; Ordered greater than
%fne = fcmp une float %x, %y  ; Unordered or not equal
```

Memory operations:

```
%ptr = alloca i32             ; Stack allocation
store i32 42, ptr %ptr        ; Write to memory
%val = load i32, ptr %ptr     ; Read from memory

; Pointer arithmetic with getelementptr (GEP)
%array_ptr = getelementptr [10 x i32], ptr %base, i64 0, i64 3
; Access element 3 of array at %base
```

Type conversion instructions:

```
%ext = zext i32 %a to i64     ; Zero-extend to wider type
%trunc = trunc i64 %b to i32  ; Truncate to narrower type
%fp_to_int = fptosi float %x to i32  ; Float to signed int
%int_to_fp = sitofp i32 %y to float  ; Signed int to float
%bitcast = bitcast i32 %z to float   ; Reinterpret bits
```

Function calls:

```
%result = call i32 @compute(i32 %x, i32 %y)
call void @print_value(i32 %result)
```

Vector operations (SIMD):

```
%vec_a = <4 x float> <float 1.0, float 2.0, float 3.0, float 4.0>
%vec_b = <4 x float> <float 5.0, float 6.0, float 7.0, float 8.0>
%vec_sum = fadd <4 x float> %vec_a, %vec_b
```

1.4.6.3 Terminator Instructions

Every basic block must end with exactly one terminator that transfers control to another block or exits the function. The terminator determines where execution continues after the current block completes. The complete list of terminator instructions is:

- `ret <type> <value>` — Return from function with a value
- `ret void` — Return from function without a value
- `br label %dest` — Unconditional jump to a basic block
- `br i1 %cond, label %true, label %false` — Conditional branch based on boolean condition
- `switch <type> <value>, label <default> [ <cases> ]` — Multi-way branch (jump table) that dispatches based on integer value
- `indirectbr <type> <address>, [ <labels> ]` — Indirect branch through a computed address (used for computed gotos and jump tables)
- `invoke <return_type> <function> (<args>) to label %normal unwind label %exception` — Function call with exception handling support; transfers to `%normal` on success or `%exception` on exception
- `resume <type> <value>` — Resume exception propagation (used in exception handling to re-throw)
- `catchswitch within <parent> [ <handlers> ] unwind <default>` — Exception catch dispatch that transfers control to appropriate catch handler
- `catchret from <catchpad> to label <successor>` — Return from catch handler to normal control flow
- `cleanupret from <cleanuppad> unwind <destination>` — Return from cleanup handler (finally block) to either normal flow or exception propagation
- `unreachable` — Marks code paths that should never be reached; enables optimizations by indicating dead code

- `callbr <return_type> <function> (<args>) to label %fallthrough [<indirect_labels>]` — Inline assembly call with multiple possible successors (supports `asm goto`)

The first basic block in every function is conventionally named `entry` and serves as the function's entry point.

1.4.7 The PHI Instruction

When control flow merges from multiple basic blocks, SSA form requires a mechanism to select the appropriate value based on which predecessor block was executed. The `phi` instruction serves this purpose.

```
define i32 @abs(i32 %x) {
entry:
  %is_neg = icmp slt i32 %x, 0
  br i1 %is_neg, label %negate, label %done

negate:
  %neg = sub i32 0, %x
  br label %done

done:
  %result = phi i32 [ %x, %entry ], [ %neg, %negate ]
  ret i32 %result
}
```

The `phi` instruction syntax is:

```
%result = phi <type> [ <value1>, %block1 ], [ <value2>, %block2 ], ...
```

Semantics: "If control flow arrived from `%block1`, use `<value1>`; if from `%block2`, use `<value2>`."

Why `phi` is necessary:

Without `phi`, we cannot maintain SSA form when values merge from different control paths. Consider this invalid attempt:

```
; INVALID: Cannot assign to %result in multiple blocks
entry:
  %result = ... ; First assignment
  br ...

other_block:
  %result = ... ; ERROR: violates SSA (redefinition)
```

The `phi` instruction solves this by creating a single assignment point that selects from multiple incoming values.

Multiple `phi` nodes:

A block can have multiple `phi` instructions. All `phi` instructions in a block are conceptually executed simultaneously before any other instructions:

```
loop:
  %i = phi i32 [ 0, %entry ], [ %i_next, %loop ]
  %sum = phi i32 [ 0, %entry ], [ %sum_next, %loop ]
  %i_next = add i32 %i, 1
  %sum_next = add i32 %sum, %i
  %cond = icmp slt i32 %i_next, 10
  br i1 %cond, label %loop, label %exit
```

This example implements a loop that sums integers from 0 to 9, demonstrating how `phi` enables iterative control flow in SSA form.

1.5 LLVM Dialects and Target Backends

While LLVM IR provides a unified intermediate representation, the compiler infrastructure must ultimately generate native machine code for diverse hardware architectures. This transformation is accomplished through **target backends**—architecture-specific code generators that translate LLVM IR into assembly or machine code for particular processors. In some contexts, especially when working with domain-specific extensions or GPU programming, these backends are referred to as **LLVM dialects**—variations of LLVM IR with specialized intrinsics, address space semantics, or instruction sets tailored to specific hardware capabilities.

1.5.1 What Are LLVM Backends and Dialects?

An LLVM **backend** is a collection of target-specific components responsible for:

- **Instruction selection**: Mapping LLVM IR instructions to target machine instructions
- **Register allocation**: Assigning virtual registers to physical hardware registers
- **Instruction scheduling**: Reordering instructions to minimize pipeline stalls and maximize throughput
- **Code emission**: Generating assembly or object files in the target's native format

The term **dialect** is often used when the IR itself is extended or constrained for a specific domain. For example, NVVM IR is technically a *dialect* of LLVM IR—it follows LLVM syntax but adds GPU-specific intrinsics and imposes restrictions on certain features (e.g., address space usage, function calling conventions).

1.5.2 Supported LLVM Target Architectures

As of LLVM 16, the compiler infrastructure supports a comprehensive range of instruction set architectures for CPUs, GPUs, and specialized processors [22]:

CPU Architectures

x86 (IA-32)	Intel 32-bit architecture (legacy, but still supported)
x86-64 (AMD64)	64-bit extension of x86 (Intel, AMD) — most mature backend
ARM	32-bit and 64-bit ARM architectures (AArch32, AArch64) — used in mobile devices, Apple Silicon, AWS Graviton
PowerPC	IBM PowerPC (32/64-bit) — used in older Mac hardware, embedded systems
RISC-V	Open-source RISC architecture (officially supported since LLVM 7)
MIPS	MIPS 32/64-bit architectures (routers, embedded systems)
SPARC	Oracle/Sun SPARC processors
z/Architecture	IBM mainframe architecture (SystemZ backend)
LoongArch	Chinese LoongArch architecture
Hexagon	Qualcomm Hexagon DSL processors
M68K	Motorola 68000 family (legacy support)
XCore	XMOS XCore processors

GPU and Accelerator Architectures

NVPTX/NVVM	NVIDIA GPUs via PTX (Parallel Thread Execution) intermediate assembly
AMDGPU	AMD GPUs (GCN, RDNA, CDNA architectures) via ROCm stack
AMD R600	Legacy AMD TeraScale GPUs (deprecated in recent LLVM versions)

Web and Virtual Targets

WASM	WebAssembly — enables compiled code execution in web browsers and server-side WASM runtimes
SPIR-V	Standard Portable Intermediate Representation for Vulkan and OpenCL (primarily through MLIR toolchain)

Deprecated Backends

Several backends have been removed from LLVM due to obsolete hardware or insufficient maintenance resources [22]:

- C backend (compile LLVM IR to C source code)
- Cell SPU (Sony/IBM Cell processor)
- MicroBlaze (Xilinx soft processor)
- DEC/Compaq Alpha (Alpha AXP)
- Nios2 (Intel/Altera soft processor)

A single Clang compiler binary typically contains *all* supported backends, enabling seamless cross-compilation without installing separate toolchains for each target architecture [22].

1.5.3 GPU Dialects: NVVM, AMDGPU, and SPIR-V

GPU programming introduces unique challenges compared to traditional CPU compilation: massive parallelism, specialized memory hierarchies (global, shared, local), SIMD-style execution models (warps/wavefronts), and hardware-specific features (tensor cores, ray tracing units). To address these requirements, LLVM provides specialized dialects and backends for major GPU vendors.

1.5.3.1 NVVM IR Dialect (NVIDIA GPUs)

NVVM IR is a compiler intermediate representation based on LLVM IR, specifically designed to represent GPU compute kernels such as CUDA kernels [16]. The relationship between NVVM IR and LLVM IR is formally defined as:

> *"NVVM IR is LLVM IR with a set of rules, restrictions, and conventions, plus a set of supported intrinsic functions. A program specified in NVVM IR is always a legal LLVM program. A legal LLVM program may not be a legal NVVM program."* [16]

Key Characteristics of NVVM IR:

- **Address space semantics**: NVVM enforces strict address space usage (generic, global, shared, local, constant) corresponding to CUDA memory hierarchy
- **Intrinsic functions**: Provides hundreds of intrinsics for thread indexing (`llvm.nvvm.read.ptx.sreg.tid.`
  synchronization (`llvm.nvvm.barrier0`), memory operations (`llvm.nvvm.ldg.*`), and hardware features (tensor cores, warp-level primitives)

- **Calling conventions**: Special conventions for kernel launches and device function calls
- **Metadata annotations**: Target architecture (SM version), kernel attributes (launch bounds, shared memory usage)

NVVM Dialects:
NVIDIA defines two NVVM IR dialects [16]:

1. **LLVM 7 dialect**: Based on LLVM 7.0.1 — supports older GPU architectures (Kepler, Maxwell, Pascal, Volta, Turing, Ampere)
2. **Modern dialect**: Based on LLVM 20.1.0 — supports Blackwell and later architectures with updated intrinsics and optimizations

The NVVM compiler (part of the CUDA Toolkit) translates NVVM IR into PTX (Parallel Thread Execution) assembly, which is then compiled to SASS (native GPU machine code) by the GPU driver at runtime or ahead-of-time.

Compilation Flow:

$$\texttt{CUDA C/C++} \rightarrow \texttt{NVVM IR} \rightarrow \texttt{PTX} \rightarrow \texttt{SASS (cubin)}$$

1.5.3.2 AMDGPU Backend (AMD GPUs)

The AMDGPU backend targets AMD Radeon and Instinct GPUs through the ROCm (Radeon Open Compute) platform. Unlike NVIDIA's proprietary NVVM approach, AMD provides an open-source LLVM backend integrated directly into the mainline LLVM project.

Key Characteristics of AMDGPU Backend:

- **Architecture support**: Targets GCN (Graphics Core Next), RDNA (gaming), and CDNA (compute) architectures
- **Address spaces**: Similar to NVVM, defines generic, global, LDS (Local Data Share), private, and constant address spaces
- **Intrinsics**: LLVM intrinsics for wave operations (`llvm.amdgcn.wave.*`), workgroup synchronization, atomic operations, and hardware-specific features (matrix cores)
- **Code object format**: Generates HSACO (HSA Code Object) files containing executable kernels

Compilation Flow:

$$\texttt{HIP/OpenCL C} \rightarrow \texttt{LLVM IR (AMDGPU)} \rightarrow \texttt{GCN/RDNA/CDNA ISA} \rightarrow \texttt{HSACO}$$

The AMDGPU backend is maintained as part of the official LLVM codebase, ensuring tight integration with LLVM optimization passes and consistent updates with new LLVM releases.

1.5.3.3 SPIR-V (Cross-Vendor Portable GPU Code)

SPIR-V (Standard Portable Intermediate Representation - Vulkan) is a binary intermediate language for graphics shaders and compute kernels, designed by the Khronos Group to enable cross-vendor portability for Vulkan and OpenCL [19].

Key Characteristics of SPIR-V:

- **Vendor-neutral**: Not tied to NVIDIA or AMD—runs on any Vulkan or OpenCL conformant GPU
- **Binary format**: Unlike LLVM IR (text-based), SPIR-V is a compact binary encoding
- **Graphics and compute**: Supports both graphics shaders (vertex, fragment, geometry) and compute kernels

- **MLIR toolchain**: LLVM's MLIR (Multi-Level Intermediate Representation) framework provides a SPIR-V dialect for generating SPIR-V binaries

Compilation Flow:

`GLSL/HLSL/OpenCL C` → `SPIR-V` → `Vendor-specific ISA (runtime JIT)`

SPIR-V enables "write once, run anywhere" for GPU compute, but sacrifices some vendor-specific optimizations compared to NVVM or AMDGPU.

1.5.4 Scope of This Handbook: Supported GPU Dialects

This handbook and the accompanying `euriklis-llvm-ir` TypeScript library focus on **heterogeneous computing and GPU-accelerated workloads**. To maintain practical scope and ensure robust implementation, we provide comprehensive support for the following GPU dialects:

1. **NVVM IR (NVIDIA GPUs)**: Full support for CUDA kernel generation targeting NVIDIA GPUs (Pascal, Volta, Turing, Ampere, Hopper, Blackwell architectures)
2. **AMDGPU (AMD GPUs)**: Support for ROCm-based GPU programming on AMD Radeon and Instinct GPUs
3. **SPIR-V (Cross-Vendor)**: Portable GPU code generation for Vulkan and OpenCL environments

What this means for TypeScript developers:
Using the `euriklis-llvm-ir` package, you can write TypeScript code that generates LLVM IR for these GPU targets, enabling:

- High-performance numerical computing (linear algebra, optimization, machine learning)
- Custom GPU kernels for domain-specific workloads (cryptography, signal processing, scientific simulation)
- Portable compute shaders that run across NVIDIA, AMD, and Intel GPUs (via SPIR-V)

While LLVM supports many additional architectures (CPUs, embedded processors, WASM), this handbook emphasizes GPU programming because it represents the most significant performance opportunity for computationally intensive TypeScript applications. Chapters 5 (Multi-Architecture Compilation) and 6 (GPU Programming) provide detailed guidance on generating code for each supported GPU dialect.

Future extensions: The `euriklis-llvm-ir` project welcomes community contributions for additional backends (CPU architectures, Apple Metal via AIR, specialized accelerators). See Chapter 11 for roadmap details and contribution guidelines.

Chapter 2

Why LLVM for the TypeScript and JavaScript Ecosystem?

2.1 The Rise of Performance-Critical JavaScript

JavaScript and TypeScript have evolved beyond web browsers. They now power:

- **Scientific computing**: TensorFlow.js, GPU.js, numerical libraries
- **Machine learning**: Neural network training, inference engines
- **Computer graphics**: Three.js, Babylon.js, game engines
- **Data processing**: Stream processing, ETL pipelines, analytics
- **Blockchain**: Smart contract VMs, cryptographic operations

These workloads hit JavaScript's performance ceiling. While V8, SpiderMonkey, and JavaScriptCore have excellent JIT compilers, they cannot compete with ahead-of-time compiled, hardware-specific code for compute-intensive tasks.

2.2 Why Not Just Use C++ or Rust?

The traditional approach is to write performance-critical code in C++/Rust and bind it to JavaScript via Node.js addons, WebAssembly, or FFI. This works but introduces friction:

1. Write core algorithm in C++/Rust
2. Compile for multiple platforms (Windows, macOS, Linux, maybe mobile)
3. Bind to JavaScript with FFI
4. Debug across language boundaries

This workflow has friction:

- **Separate toolchains**: npm + cargo/cmake + wasm-pack
- **Build complexity**: native compilation per platform
- **Type mismatches**: JavaScript objects $\leftrightarrow$ C structs
- **Limited GPU access**: WebGPU is new; CUDA requires native code

2.3 LLVM as the Universal Backend

LLVM provides a *universal code generation backend*:

1. **Write code generation logic in TypeScript** (type-safe, testable, version-controlled with NPM)

2. **Generate LLVM IR** (platform-independent intermediate representation)
3. **Compile to any architecture**: x86, ARM, CUDA, AMD, WASM—from the same IR
4. **Leverage LLVM's optimizer**: Auto-vectorization, loop unrolling, inlining—for free

Example workflow:

```
// TypeScript
import { LLVMCodegen } from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("my_module");
// ... build IR programmatically ...

const ir = codegen.toString();
// Emit LLVM IR as text (platform-independent)
```

Then compile with `llc` or `clang`:

```
$ llc -march=x86-64 -o output.o module.ll       # x86-64
$ llc -march=aarch64 -o output.o module.ll      # ARM
$ llc -march=nvptx64 -o output.ptx module.ll    # NVIDIA GPU
$ llc -march=amdgcn -o output.o module.ll       # AMD GPU
```

Optimization strategies: LLVM provides multiple optimization levels, controlled via the `-O` flag:

Level	Description
`-O0`	No optimization (fastest compilation, useful for debugging)
`-O1`	Basic optimizations (constant folding, dead code elimination, simple inlining)
`-O2`	Moderate optimizations (most production code uses this: loop unrolling, vectorization, function inlining)
`-O3`	Aggressive optimizations (may increase binary size; adds advanced loop transformations and aggressive inlining)
`-Os`	Optimize for size (useful for embedded systems and WASM)
`-Oz`	Aggressively optimize for size (even more than `-Os`)

To apply optimizations, you can either:

Option 1: Use `opt` before `llc`

```
$ opt -O3 module.ll -o optimized.ll    # Optimize IR
$ llc -march=x86-64 optimized.ll -o output.o
```

Option 2: Use `llc` directly with optimization level

```
$ llc -O3 -march=x86-64 -o output.o module.ll
```

Option 3: Use `clang` (combines everything)

```
$ clang -O3 -c module.ll -o output.o
```

For most production use cases, `-O2` provides an excellent balance between compilation time and runtime performance. GPU code (`nvptx64`, `amdgcn`) often benefits from `-O3` due to the massively parallel architecture.

2.4 Use Cases in the JavaScript Ecosystem

1. **DSL compilers**: Build domain-specific languages that compile to native code (query languages, shader languages, config languages)

2. **JIT compilers**: Runtime code generation for interpreters (JavaScript VMs, database query engines)
3. **GPU compute from Node.js**: Generate CUDA/ROCm kernels at runtime without writing C++ wrappers
4. **WebAssembly optimization**: Generate optimized WASM modules from high-level abstractions
5. **Cross-compilation tools**: Build deployment toolchains that target multiple architectures from one codebase

2.5 Why a TypeScript API Instead of C Bindings?

LLVM provides a comprehensive C API (`llvm-c`), and several Node.js bindings exist (`llvm-node`[15], `llvm-bindings`[10], `node-llvmc`[11]). However, FFI bindings have inherent limitations:

1. **Native dependencies**: Require compiling against specific LLVM versions
   - LLVM 15, 16, 17+ are not API-stable
   - Users must install exact LLVM version on their system
   - Cross-compilation becomes complex
2. **Platform fragility**: Native Node.js addons break across Node versions
   - Rebuilding required for Node 18, 20, 22, etc.
   - Electron and Bun compatibility issues
3. **Distribution complexity**: npm packages with native addons need pre-built binaries for every platform
4. **No tree-shaking**: FFI bundles entire LLVM library (hundreds of MB)

2.6 @euriklis/llvm-ir Design Philosophy

This package takes a different approach: **generate LLVM IR as text**. No native bindings, no FFI, no compiled dependencies.

Advantages:

- **Zero dependencies**: Pure TypeScript, runs in Node.js, Bun, Deno, browsers
- **Instant installation**: `npm install @euriklis/llvm-ir` with no compilation step
- **Type-safe API**: Full IntelliSense with TypeScript types
- **Debuggable output**: Inspect generated IR as human-readable text
- **Composable**: Pipe IR to any LLVM tool (`llc`, `opt`, `lli`)
- **Portable**: Same code works on Windows, macOS, Linux, ARM, x86

Trade-offs:

- Cannot use LLVM's optimizer programmatically (must shell out to `opt`)
- Cannot JIT-compile directly (must use `lli` or write IR to file)
- Validation happens at compile-time via external tools, not runtime

For most use cases—code generation, DSL compilers, build tools—these trade-offs are acceptable. The benefits of simplicity, portability, and zero native dependencies far outweigh the limitations.

2.7 Summary

LLVM provides a universal compilation backend. By generating LLVM IR from TypeScript:

- You stay in the JavaScript ecosystem (familiar tools, npm, TypeScript types)
- You unlock native performance on CPUs and GPUs
- You avoid platform-specific build complexity
- You leverage decades of LLVM optimization research

The rest of this handbook shows you how.

Part II

Getting Started with LLVM IR

Chapter 3

Installation and Setup

3.1 Installation

```
npm install @euriklis/llvm-ir
```

Or with Bun:

```
bun add @euriklis/llvm-ir
```

That's it. No native dependencies, no compilation step, no system LLVM installation required (though you'll want LLVM's `llc` compiler to generate binaries).

Chapter 4

Implementing TypeScript Functions as LLVM IR

4.1 Introduction

This chapter demonstrates how to translate TypeScript logic into LLVM IR using the library's object-based API. Each example shows the TypeScript concept, the generated IR, and the underlying assembly/execution model.

4.2 Example 1: Simple Arithmetic

TypeScript equivalent:

```
function add(a: number, b: number): number {
  return a + b;
}
```

LLVM IR generation:

```
import {
  LLVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  BinaryInst,
  RetInst
} from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("arithmetic");

const addFunc = new LLVMFunction({
  name: "add",
  returnType: LLVMCodegen.types.i32
});

addFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "a"
}));
addFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "b"
}));
```

```
const entry = new BasicBlock("entry");
entry.addInstruction(new BinaryInst({
  name: "result",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: "a",
  rhs: "b"
}));
entry.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "result"
}));

addFunc.addBasicBlock(entry);
codegen.getModule().addFunction(addFunc);

console.log(codegen.toString());
```

Generated LLVM IR:

```
target triple = "x86_64-unknown-linux-gnu"

define i32 @add(i32 %a, i32 %b) {
entry:
  %result = add i32 %a, %b
  ret i32 %result
}
```

Key concepts:

- `LLVMFunction`: Defines function signature
- `Parameter`: Typed function argument (automatically prefixed with `%`)
- `BasicBlock`: Container for instructions (must end with terminator)
- `BinaryInst`: Arithmetic operation (add, sub, mul, etc.)
- `RetInst`: Return value (terminates basic block)

4.3 Example 2: Conditional Logic (if-else)

TypeScript equivalent:

```
function max(a: number, b: number): number {
  if (a > b) {
    return a;
  } else {
    return b;
  }
}
```

LLVM IR generation with control flow:

```
import {
  LLVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  ICmpInst,
  BrInst,
  RetInst
```

```
} from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("conditional");

const maxFunc = new LLVMFunction({
  name: "max",
  returnType: LLVMCodegen.types.i32
});

maxFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "a"
}));
maxFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "b"
}));

const entry = new BasicBlock("entry");
const ifThen = new BasicBlock("if.then");
const ifElse = new BasicBlock("if.else");

// entry: compare a > b
entry.addInstruction(new ICmpInst({
  name: "cmp",
  predicate: LLVMCodegen.predicates.sgt,  // signed greater than
  type: LLVMCodegen.types.i32,
  lhs: "a",
  rhs: "b"
}));
entry.setTerminator(new BrInst({
  condition: "cmp",
  conditionType: LLVMCodegen.types.i1,
  target: ifThen,
  falseTarget: ifElse
}));

// if.then: return a
ifThen.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "a"
}));

// if.else: return b
ifElse.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "b"
}));

maxFunc.addBasicBlock(entry);
maxFunc.addBasicBlock(ifThen);
maxFunc.addBasicBlock(ifElse);
codegen.getModule().addFunction(maxFunc);

console.log(codegen.toString());
```

Generated LLVM IR:

```
define i32 @max(i32 %a, i32 %b) {
```

```
entry:
  %cmp = icmp sgt i32 %a, %b
  br i1 %cmp, label %if.then, label %if.else

if.then:
  ret i32 %a

if.else:
  ret i32 %b
}
```

Key concepts:

- `ICmpInst`: Integer comparison (returns `i1` boolean)
- `BrInst`: Conditional branch (if-else in LLVM)
- Multiple basic blocks: Each path gets its own block
- Control flow graph: `entry` → `if.then` or `if.else`

4.4 Example 3: Loops with PHI Nodes

TypeScript equivalent:

```
function sum(n: number): number {
  let total = 0;
  for (let i = 0; i < n; i++) {
    total += i;
  }
  return total;
}
```

LLVM IR generation with loops:

```
import {
  LLVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  PhiInst,
  ICmpInst,
  BinaryInst,
  BrInst,
  RetInst
} from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("loop");

const sumFunc = new LLVMFunction({
  name: "sum",
  returnType: LLVMCodegen.types.i32
});

sumFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "n"
}));

const entry = new BasicBlock("entry");
const loop = new BasicBlock("loop");
```

```
const exit = new BasicBlock("exit");

// entry: unconditional jump to loop
entry.setTerminator(new BrInst({ target: loop }));

// loop: PHI nodes for i and total
loop.addInstruction(new PhiInst({
  name: "i",
  type: LLVMCodegen.types.i32,
  incomingValues: [
    { value: 0, block: entry },
    { value: "i.next", block: loop }
  ]
}));

loop.addInstruction(new PhiInst({
  name: "total",
  type: LLVMCodegen.types.i32,
  incomingValues: [
    { value: 0, block: entry },
    { value: "total.next", block: loop }
  ]
}));

// total.next = total + i
loop.addInstruction(new BinaryInst({
  name: "total.next",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: "total",
  rhs: "i"
}));

// i.next = i + 1
loop.addInstruction(new BinaryInst({
  name: "i.next",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: "i",
  rhs: 1
}));

// cmp = (i.next < n)
loop.addInstruction(new ICmpInst({
  name: "cmp",
  predicate: LLVMCodegen.predicates.slt,
  type: LLVMCodegen.types.i32,
  lhs: "i.next",
  rhs: "n"
}));

// if cmp, continue loop; else exit
loop.setTerminator(new BrInst({
  condition: "cmp",
  conditionType: LLVMCodegen.types.i1,
  target: loop,
  falseTarget: exit
}));
```

```
// exit: return total
exit.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "total"
}));

sumFunc.addBasicBlock(entry);
sumFunc.addBasicBlock(loop);
sumFunc.addBasicBlock(exit);
codegen.getModule().addFunction(sumFunc);

console.log(codegen.toString());
```

Generated LLVM IR:

```
define i32 @sum(i32 %n) {
entry:
  br label %loop

loop:
  %i = phi i32 [ 0, %entry ], [ %i.next, %loop ]
  %total = phi i32 [ 0, %entry ], [ %total.next, %loop ]
  %total.next = add i32 %total, %i
  %i.next = add i32 %i, 1
  %cmp = icmp slt i32 %i.next, %n
  br i1 %cmp, label %loop, label %exit

exit:
  ret i32 %total
}
```

Key concepts:

- `PhiInst`: Merge values from different control flow paths
- Loop structure: `entry` → `loop` → (`loop` or `exit`)
- SSA form maintained: `i`, `i.next`, `total`, `total.next` are all unique
- Back-edge: `loop` can branch to itself

4.5 Type Shortcuts for Cleaner Code

Throughout this handbook, we use type shortcuts for brevity:

```
import { LLVMCodegen } from "@euriklis/llvm-ir";

// Destructure common types
const { i32, i64, float, double, ptr, void: voidType } = LLVMCodegen.
   types;

// Now use directly
const add = new BinaryInst({
  name: "sum",
  opcode: LLVMCodegen.opcodes.add,
  type: i32,  // Instead of LLVMCodegen.types.i32
  lhs: "a",
  rhs: "b"
});
```

These are equivalent to their fully-qualified names:

- `i32` = `LLVMCodegen.types.i32`
- `float` = `LLVMCodegen.types.float`
- `ptr` = `LLVMCodegen.types.ptr`

Part III

Multi-Architecture Compilation

Chapter 5

Compiling LLVM IR for Different CPU Architectures

5.1 Target Triples

One of LLVM IR's key strengths is **write once, compile anywhere**. The same IR can target x86-64, ARM, RISC-V, or WebAssembly by changing the compilation flags.

A target triple specifies the architecture, vendor, and operating system:

```
import { LLVMCodegen } from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("my_module");
codegen.targetTriple = "x86_64-unknown-linux-gnu";  // x86-64 Linux
codegen.dataLayout = "e-m:e-p270:32:32-i64:64-f80:128-n8:16:32:64-S128
    ";
```

Common triples:

```
x86_64-unknown-linux-gnu        # Linux x86-64
x86_64-apple-darwin             # macOS Intel
aarch64-unknown-linux-gnu       # Linux ARM64 (Raspberry Pi, AWS
    Graviton)
aarch64-apple-darwin            # macOS Apple Silicon (M1/M2/M3)
armv7-unknown-linux-gnueabihf   # ARM 32-bit (Raspberry Pi 3)
riscv64-unknown-linux-gnu       # RISC-V 64-bit
wasm32-unknown-unknown          # WebAssembly 32-bit
wasm64-unknown-unknown          # WebAssembly 64-bit
```

5.2 Compiling with llc

Once you have LLVM IR, compile it with `llc` (LLVM static compiler):

```
# Generate x86-64 assembly
llc -march=x86-64 -filetype=asm -o output.s module.ll

# Generate ARM64 object file
llc -march=aarch64 -filetype=obj -o output.o module.ll

# Generate RISC-V binary
llc -march=riscv64 -filetype=obj -o output.o module.ll

# Generate WebAssembly
llc -march=wasm32 -filetype=obj -o output.wasm module.ll
```

Or use `clang` to link directly:

```
clang -o program module.ll
./program
```

5.3 Architecture-Specific Code with this Library

The `@euriklis/llvm-ir` package provides architecture-specific code generators:

x86-64 (default):

```
import { LLVMCodegen } from "@euriklis/llvm-ir";
const codegen = new LLVMCodegen("x86_module");
// Targets x86-64 by default
```

ARM/AArch64:

```
import { ARMCodegen } from "@euriklis/llvm-ir";
const codegen = new ARMCodegen("arm_module", true);  // true = 64-bit
// Generates: target triple = "aarch64-unknown-linux-gnu"
```

RISC-V:

```
import { RISCVCodegen } from "@euriklis/llvm-ir";
const codegen = new RISCVCodegen("riscv_module");
// Generates: target triple = "riscv64-unknown-elf"
```

WebAssembly:

```
import { WASMCodegen } from "@euriklis/llvm-ir";
const codegen = new WASMCodegen("wasm_module");
// Generates: target triple = "wasm32-unknown-unknown"
```

All use the same API—only the code generator class changes.

5.4 Data Layout Strings

The data layout string specifies memory alignment, pointer sizes, and endianness:

```
e-m:e-p270:32:32-i64:64-f80:128-n8:16:32:64-S128
```

Breakdown:

- `e`: Little-endian (Intel, ARM). `E` would be big-endian.
- `m:e`: ELF mangling (Linux).
- `p270:32:32`: Pointers in address space 270 are 32-bit with 32-bit alignment.
- `i64:64`: 64-bit integers have 64-bit alignment.
- `f80:128`: 80-bit floats (x87) have 128-bit alignment.
- `n8:16:32:64`: Native integer widths (CPU can operate on these efficiently).
- `S128`: Stack alignment is 128 bits (16 bytes).

Each architecture has its own data layout. The library sets these automatically when you use architecture-specific code generators.

5.5 Cross-Compilation Example

Same TypeScript code, three architectures:

```
import {
  LLVMCodegen,
  ARMCodegen,
  WASMCodegen,
  LLVMFunction,
  BasicBlock,
  RetInst
} from "@euriklis/llvm-ir";

function createAddFunction(codegen) {
  const func = new LLVMFunction({
    name: "add",
    returnType: LLVMCodegen.types.i32
  });
  // ... (same function body for all architectures)
  codegen.getModule().addFunction(func);
}

// x86-64
const x86 = new LLVMCodegen("x86");
createAddFunction(x86);
console.log("=== x86-64 ===");
console.log(x86.toString());

// ARM64
const arm = new ARMCodegen("arm", true);
createAddFunction(arm);
console.log("=== ARM64 ===");
console.log(arm.toString());

// WebAssembly
const wasm = new WASMCodegen("wasm");
createAddFunction(wasm);
console.log("=== WASM ===");
console.log(wasm.toString());
```

Each will have a different target triple, but the function IR is identical. LLVM's backends handle architecture-specific optimizations (register allocation, instruction selection, calling conventions).

Chapter 6

GPU Programming: Same Function Across All Architectures

6.1 Introduction

LLVM supports GPU targets via architecture-specific backends:

- **NVIDIA**: NVPTX backend (CUDA)
- **AMD**: AMDGPU backend (ROCm)
- **Intel/OpenCL**: SPIR-V backend

This chapter shows how to write the same algorithm (vector addition) for all three GPU architectures using this library's unified API.

6.2 Vector Addition: NVIDIA CUDA

CUDA C equivalent:

```
__global__ void vectorAdd(float* a, float* b, float* c, int n) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < n) {
    c[idx] = a[idx] + b[idx];
  }
}
```

TypeScript with NVVMCodegen:

```
import {
  NVVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  BinaryInst,
  ICmpInst,
  BrInst,
  LoadInst,
  StoreInst,
  GetElementPtrInst,
  RetInst,
  LLVMCodegen,
  FloatType
} from "@euriklis/llvm-ir";
```

```
const codegen = new NVVMCodegen("vector_add");
const module = codegen.getModule();

const floatType = new FloatType("float");

const kernel = new LLVMFunction({
  name: "vectorAdd",
  returnType: LLVMCodegen.types.void
});

kernel.addParameter(new Parameter({ type: LLVMCodegen.types.ptr, name:
   "a" }));
kernel.addParameter(new Parameter({ type: LLVMCodegen.types.ptr, name:
   "b" }));
kernel.addParameter(new Parameter({ type: LLVMCodegen.types.ptr, name:
   "c" }));
kernel.addParameter(new Parameter({ type: LLVMCodegen.types.i32, name:
   "n" }));

const entry = new BasicBlock("entry");
const compute = new BasicBlock("compute");
const exit = new BasicBlock("exit");

// Get thread ID using CUDA intrinsics
const blockIdx = NVVMCodegen.cuda.blockIdxX("blockIdx.x");
const blockDim = NVVMCodegen.cuda.blockDimX("blockDim.x");
const threadIdx = NVVMCodegen.cuda.threadIdxX("threadIdx.x");

entry.addInstruction(blockIdx);
entry.addInstruction(blockDim);
entry.addInstruction(threadIdx);

// idx = blockIdx.x * blockDim.x + threadIdx.x
const mul = new BinaryInst({
  name: "mul",
  opcode: LLVMCodegen.opcodes.mul,
  type: LLVMCodegen.types.i32,
  lhs: blockIdx,
  rhs: blockDim
});
entry.addInstruction(mul);

const idx = new BinaryInst({
  name: "idx",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: mul,
  rhs: threadIdx
});
entry.addInstruction(idx);

// Bounds check: idx < n
const cmp = new ICmpInst({
  name: "cmp",
  predicate: LLVMCodegen.predicates.slt,
  type: LLVMCodegen.types.i32,
  lhs: idx,
  rhs: "n"
```

```
});
entry.addInstruction(cmp);

entry.setTerminator(new BrInst({
  condition: cmp,
  conditionType: LLVMCodegen.types.i1,
  target: compute,
  falseTarget: exit
}));

// Compute: c[idx] = a[idx] + b[idx]
// ... (full implementation in repository examples)

kernel.addBasicBlock(entry);
kernel.addBasicBlock(compute);
kernel.addBasicBlock(exit);
module.addFunction(kernel);

console.log(codegen.toString());
```

Key CUDA intrinsics:

- `NVVMCodegen.cuda.threadIdxX/Y/Z`: Thread index within block
- `NVVMCodegen.cuda.blockIdxX/Y/Z`: Block index within grid
- `NVVMCodegen.cuda.blockDimX/Y/Z`: Block dimensions
- `NVVMCodegen.cuda.syncThreads()`: Barrier synchronization

6.3 Vector Addition: AMD ROCm

HIP equivalent (AMD's CUDA-like API):

```
__global__ void vectorAdd(float* a, float* b, float* c, int n) {
  int idx = hipBlockIdx_x * hipBlockDim_x + hipThreadIdx_x;
  if (idx < n) {
    c[idx] = a[idx] + b[idx];
  }
}
```

TypeScript with AMDGPUCodegen:

```
import { AMDGPUCodegen } from "@euriklis/llvm-ir";

const codegen = new AMDGPUCodegen("vector_add");

// Same structure as CUDA, but use AMD intrinsics:
const workitemId = AMDGPUCodegen.rocm.workitemIdX("tid.x");
const workgroupId = AMDGPUCodegen.rocm.workgroupIdX("gid.x");
// Workgroup size is typically passed as kernel parameter

// Rest identical to CUDA version
```

Key AMD intrinsics:

- `AMDGPUCodegen.rocm.workitemIdX/Y/Z`: Work-item ID (like `threadIdx`)
- `AMDGPUCodegen.rocm.workgroupIdX/Y/Z`: Work-group ID (like `blockIdx`)
- `AMDGPUCodegen.rocm.barrier()`: Work-group barrier

6.4 Vector Addition: Intel GPU (OpenCL)

OpenCL kernel:

```
__kernel void vectorAdd(__global float* a, __global float* b,
                        __global float* c, int n) {
  int idx = get_global_id(0);
  if (idx < n) {
    c[idx] = a[idx] + b[idx];
  }
}
```

TypeScript with SPIRVCodegen:

```
import { SPIRVCodegen } from "@euriklis/llvm-ir";

const codegen = new SPIRVCodegen("vector_add");

// OpenCL provides global ID directly
const globalId = SPIRVCodegen.opencl.getGlobalIdX("gid");
// Note: Returns i64, may need ZExtInst for conversions

// Rest similar to CUDA/AMD versions
```

Key OpenCL intrinsics:

- `SPIRVCodegen.opencl.getGlobalIdX/Y/Z`: Global work-item ID (i64)
- `SPIRVCodegen.opencl.getLocalIdX/Y/Z`: Local work-item ID
- `SPIRVCodegen.opencl.getGroupIdX/Y/Z`: Work-group ID
- `SPIRVCodegen.opencl.barrier()`: Work-group barrier

6.5 GPU Architecture Comparison

Concept	**NVIDIA (NVPTX)**	**AMD (AMDGPU)**	**Intel (SPIR-V)**
Thread ID	`threadIdx`	`workitemId`	`get_local_id`
Block/Group ID	`blockIdx`	`workgroupId`	`get_group_id`
Block/Group Size	`blockDim`	(parameter)	`get_local_size`
Global ID	`blockIdx * blockDim + threadIdx`	(computed)	`get_global_id`
SIMD Width	Warp (32)	Wavefront (64)	Sub-group (8-32)
Synchronization	`__syncthreads`	`barrier`	`barrier`
Shared Memory	`__shared__` (addr space 3)	`__local` (addr space 3)	`__local` (addr space 3)

All three dialects are production-ready with comprehensive intrinsic libraries and `llc`-validated IR generation. See the repository's `src/` directory for complete working examples.

6.6 Compiling GPU LLVM IR to Machine Code

After generating LLVM IR with GPU-specific intrinsics, you must compile it to the target GPU's native format using `llc` or vendor-specific tools.

6.6.1 NVIDIA: Compiling NVVM IR to PTX

PTX is NVIDIA's intermediate assembly language for CUDA. It's then JIT-compiled by the GPU driver to native SASS for your specific GPU architecture.

Using llc to generate PTX:

```
# Basic PTX generation
llc -march=nvptx64 -mcpu=sm_75 -o kernel.ptx module.ll

# With optimizations (recommended)
llc -O3 -march=nvptx64 -mcpu=sm_75 -o kernel.ptx module.ll
```

Common NVIDIA compute capabilities (-mcpu):

- `sm_50`: Maxwell (GTX 900 series)
- `sm_60`: Pascal (GTX 10xx, Tesla P100)
- `sm_70`: Volta (Tesla V100)
- `sm_75`: Turing (RTX 20xx, T4)
- `sm_80`: Ampere (A100, RTX 30xx)
- `sm_86`: Ampere (RTX 30xx mobile)
- `sm_89`: Ada Lovelace (RTX 40xx)
- `sm_90`: Hopper (H100)

Loading PTX from Node.js:

```
import { readFileSync } from "fs";

// Assuming you have a CUDA runtime binding (e.g., gpu.js, node-cuda)
const ptxCode = readFileSync("kernel.ptx", "utf-8");
// Load PTX module and launch kernel with CUDA runtime
```

6.6.2 AMD: Compiling AMDGPU IR to GCN/RDNA

AMD GPUs use **GCN** or **RDNA** instruction sets. The LLVM AMDGPU backend can generate both object files and assembly.

Using llc for AMD GPUs:

```
# Generate AMD GCN assembly
llc -march=amdgcn -mcpu=gfx900 -o kernel.s module.ll

# Generate object file (for ROCm linking)
llc -O3 -march=amdgcn -mcpu=gfx1030 -filetype=obj -o kernel.o module.ll
```

Common AMD GPU architectures (-mcpu):

- `gfx803`: Polaris (RX 400/500 series)
- `gfx900`: Vega (Radeon VII, MI25)
- `gfx906`: Vega 20 (MI50, MI60)
- `gfx908`: CDNA (MI100)
- `gfx90a`: CDNA2 (MI200 series)
- `gfx1030`: RDNA2 (RX 6000 series)
- `gfx1100`: RDNA3 (RX 7000 series)

Integration with ROCm:

```
# Link with ROCm device libraries
clang -target amdgcn-amd-amdhsa -mcpu=gfx906 \
      -o kernel.co module.ll
```

```
# Create HSA code object
clang-offload-bundler --type=o \
      --targets=hipv4-amdgcn-amd-amdhsa--gfx906 \
      --inputs=kernel.co --outputs=kernel.hsaco
```

6.6.3 Intel/OpenCL: Compiling to SPIR-V Binary

SPIR-V is Khronos's standard intermediate representation for GPU compute (OpenCL, Vulkan).

Using llvm-spirv translator:

```
# First, compile to LLVM bitcode
llc -march=spir64 -filetype=obj -o module.bc module.ll

# Then translate to SPIR-V binary
llvm-spirv module.bc -o kernel.spv
```

Alternative: Direct SPIR-V generation with clang:

```
clang -cc1 -triple spir64-unknown-unknown \
      -emit-llvm-bc -o module.bc module.ll

llvm-spirv module.bc -o kernel.spv
```

Loading SPIR-V in OpenCL:

```
import { readFileSync } from "fs";

// Using an OpenCL binding for Node.js
const spirvBinary = readFileSync("kernel.spv");
// Create OpenCL program from SPIR-V binary
// const program = clCreateProgramWithIL(context, spirvBinary, ...);
```

6.6.4 GPU Compilation Workflow Summary

GPU Vendor	Output Format	Compilation Command
NVIDIA	PTX (text)	`llc -march=nvptx64 -mcpu=sm_XX -o out.ptx`
AMD	GCN assembly or object	`llc -march=amdgcn -mcpu=gfxXXXX -o out.s`
Intel/OpenCL	SPIR-V binary	`llc -march=spir64 | llvm-spirv -o out.spv`

Key differences:

- **NVIDIA PTX** is human-readable text (like assembly), JIT-compiled by the driver at runtime
- **AMD GCN/RDNA** can be assembly (.s) or binary (.o), linked with ROCm runtime
- **SPIR-V** is a binary format, consumed by OpenCL/Vulkan drivers

Always use `-O3` optimization for GPU code—the parallel nature of GPU execution benefits heavily from loop unrolling, vectorization, and memory coalescing optimizations.

Part IV

Reference

Chapter 7

Instruction Reference

7.1 Introduction

This chapter provides a comprehensive reference for all LLVM IR instructions supported by the library.

7.2 Arithmetic Operations

Integer arithmetic:

Instruction	TypeScript	LLVM IR
Addition	`BinaryInst{opcode: "add"}`	`%r = add i32 %a, %b`
Subtraction	`BinaryInst{opcode: "sub"}`	`%r = sub i32 %a, %b`
Multiplication	`BinaryInst{opcode: "mul"}`	`%r = mul i32 %a, %b`
Division (signed)	`BinaryInst{opcode: "sdiv"}`	`%r = sdiv i32 %a, %b`
Division (unsigned)	`BinaryInst{opcode: "udiv"}`	`%r = udiv i32 %a, %b`
Remainder (signed)	`BinaryInst{opcode: "srem"}`	`%r = srem i32 %a, %b`
Remainder (unsigned)	`BinaryInst{opcode: "urem"}`	`%r = urem i32 %a, %b`

Floating-point arithmetic:

Instruction	TypeScript	LLVM IR
Addition	`BinaryInst{opcode: "fadd"}`	`%r = fadd float %a, %b`
Subtraction	`BinaryInst{opcode: "fsub"}`	`%r = fsub float %a, %b`
Multiplication	`BinaryInst{opcode: "fmul"}`	`%r = fmul float %a, %b`
Division	`BinaryInst{opcode: "fdiv"}`	`%r = fdiv float %a, %b`

Remainder	`BinaryInst{opcode: "frem"}`	`%r = frem float %a, %b`

7.3 Bitwise Operations

Instruction	TypeScript	LLVM IR
Bitwise AND	`BinaryInst{opcode: "and"}`	`%r = and i32 %a, %b`
Bitwise OR	`BinaryInst{opcode: "or"}`	`%r = or i32 %a, %b`
Bitwise XOR	`BinaryInst{opcode: "xor"}`	`%r = xor i32 %a, %b`
Shift left	`BinaryInst{opcode: "shl"}`	`%r = shl i32 %a, %b`
Logical shift right	`BinaryInst{opcode: "lshr"}`	`%r = lshr i32 %a, %b`
Arithmetic shift right	`BinaryInst{opcode: "ashr"}`	`%r = ashr i32 %a, %b`

7.4 Comparison Operations

Integer comparisons (ICmpInst):

Predicate	Meaning	Example
`eq`	Equal	`%r = icmp eq i32 %a, %b`
`ne`	Not equal	`%r = icmp ne i32 %a, %b`
`sgt`	Signed greater	`%r = icmp sgt i32 %a, %b`
`sge`	Signed greater/equal	`%r = icmp sge i32 %a, %b`
`slt`	Signed less	`%r = icmp slt i32 %a, %b`
`sle`	Signed less/equal	`%r = icmp sle i32 %a, %b`
`ugt`	Unsigned greater	`%r = icmp ugt i32 %a, %b`
`uge`	Unsigned greater/equal	`%r = icmp uge i32 %a, %b`
`ult`	Unsigned less	`%r = icmp ult i32 %a, %b`
`ule`	Unsigned less/equal	`%r = icmp ule i32 %a, %b`

Floating-point comparisons (FCmpInst):

Predicate	Meaning	Example
`oeq`	Ordered equal	`%r = fcmp oeq float %a, %b`
`one`	Ordered not equal	`%r = fcmp one float %a, %b`
`ogt`	Ordered greater	`%r = fcmp ogt float %a, %b`
`oge`	Ordered greater/equal	`%r = fcmp oge float %a, %b`
`olt`	Ordered less	`%r = fcmp olt float %a, %b`
`ole`	Ordered less/equal	`%r = fcmp ole float %a, %b`

uno	Unordered (NaN)	`%r = fcmp uno float %a, %b`

7.5 Memory Operations

Stack allocation:

```
import { AllocaInst } from "@euriklis/llvm-ir";

const alloc = new AllocaInst({
  name: "ptr",
  allocatedType: LLVMCodegen.types.i32,
  align: 4
});
// Generates: %ptr = alloca i32, align 4
```

Load from memory:

```
import { LoadInst } from "@euriklis/llvm-ir";

const load = new LoadInst({
  name: "val",
  type: LLVMCodegen.types.i32,
  pointerType: LLVMCodegen.types.ptr,
  pointer: "ptr",
  align: 4
});
// Generates: %val = load i32, ptr %ptr, align 4
```

Store to memory:

```
import { StoreInst } from "@euriklis/llvm-ir";

const store = new StoreInst({
  valueType: LLVMCodegen.types.i32,
  value: 42,
  pointerType: LLVMCodegen.types.ptr,
  pointer: "ptr",
  align: 4
});
// Generates: store i32 42, ptr %ptr, align 4
```

Array/struct indexing (GetElementPtr):

```
import { GetElementPtrInst } from "@euriklis/llvm-ir";

const gep = new GetElementPtrInst({
  name: "elem_ptr",
  baseType: LLVMCodegen.types.i32,
  ptrType: LLVMCodegen.types.ptr,
  pointer: "array",
  indices: [
    { type: LLVMCodegen.types.i32, value: 0 },
    { type: LLVMCodegen.types.i32, value: "idx" }
  ]
});
// Generates: %elem_ptr = getelementptr i32, ptr %array, i32 0, i32 %
   idx
```

7.6 Control Flow

Conditional branch:

```
import { BrInst } from "@euriklis/llvm-ir";

const br = new BrInst({
  condition: "cmp",
  conditionType: LLVMCodegen.types.i1,
  target: thenBlock,
  falseTarget: elseBlock
});
// Generates: br i1 %cmp, label %then, label %else
```

Unconditional branch:

```
const br = new BrInst({ target: nextBlock });
// Generates: br label %next
```

Return:

```
import { RetInst } from "@euriklis/llvm-ir";

// Return value
const ret = new RetInst({
  type: LLVMCodegen.types.i32,
  value: 42
});
// Generates: ret i32 42

// Return void
const retVoid = new RetInst({});
// Generates: ret void
```

Switch statement:

```
import { SwitchInst } from "@euriklis/llvm-ir";

const sw = new SwitchInst({
  value: "x",
  valueType: LLVMCodegen.types.i32,
  defaultDest: defaultBlock,
  cases: [
    { value: 0, dest: case0Block },
    { value: 1, dest: case1Block },
    { value: 2, dest: case2Block }
  ]
});
// Generates: switch i32 %x, label %default [
//              i32 0, label %case0
//              i32 1, label %case1
//              i32 2, label %case2
//            ]
```

7.7 Type Conversions

Instruction	Purpose	Example

`TruncInst`	Truncate integer	`%r = trunc i64 %a to i32`
`ZExtInst`	Zero-extend integer	`%r = zext i32 %a to i64`
`SExtInst`	Sign-extend integer	`%r = sext i32 %a to i64`
`FPTruncInst`	Truncate float	`%r = fptrunc double %a to float`
`FPExtInst`	Extend float	`%r = fpext float %a to double`
`BitCastInst`	Reinterpret bits	`%r = bitcast i32 %a to float`

7.8 Function Calls

```
import { CallInst } from "@euriklis/llvm-ir";

const call = new CallInst({
  name: "result",
  returnType: LLVMCodegen.types.i32,
  functionName: "my_function",
  args: [
    { type: LLVMCodegen.types.i32, value: 10 },
    { type: LLVMCodegen.types.float, value: "x" }
  ]
});
// Generates: %result = call i32 @my_function(i32 10, float %x)
```

7.9 Vector Operations

```
import {
  InsertElementInst,
  ExtractElementInst,
  ShuffleVectorInst,
  VectorType
} from "@euriklis/llvm-ir";

const vec4 = new VectorType({ elementType: float, count: 4 });

// Insert element
const insert = new InsertElementInst({
  name: "v2",
  vectorType: vec4,
  vector: "v1",
  element: 3.14,
  elementType: float,
  index: 0,
  indexType: i32
});
// Generates: %v2 = insertelement <4 x float> %v1, float 3.14, i32 0

// Extract element
const extract = new ExtractElementInst({
  name: "elem",
  vectorType: vec4,
  vector: "v",
```

```
  index: 2,
  indexType: i32
});
// Generates: %elem = extractelement <4 x float> %v, i32 2

// Shuffle vector
const shuffle = new ShuffleVectorInst({
  name: "result",
  vectorType: vec4,
  vector1: "v1",
  vector2: "v2",
  mask: [0, 1, 6, 7]
});
// Generates: %result = shufflevector <4 x float> %v1, <4 x float> %v2,
//                                    <4 x i32> <i32 0, i32 1, i32 6,
    i32 7>
```

7.10 Atomic Operations

```
import { AtomicRMWInst, AtomicCmpXchgInst, FenceInst } from "@euriklis/
    llvm-ir";

// Atomic read-modify-write
const atomicAdd = new AtomicRMWInst({
  name: "old_val",
  operation: "add",
  pointer: "counter",
  pointerType: ptr,
  value: 1,
  valueType: i32,
  ordering: "seq_cst"
});
// Generates: %old_val = atomicrmw add ptr %counter, i32 1 seq_cst

// Compare-and-swap
const cas = new AtomicCmpXchgInst({
  name: "result",
  pointer: "lock",
  pointerType: ptr,
  cmp: 0,
  new: 1,
  valueType: i32,
  successOrdering: "seq_cst",
  failureOrdering: "acquire"
});
// Generates: %result = cmpxchg ptr %lock, i32 0, i32 1 seq_cst acquire

// Memory fence
const fence = new FenceInst({ ordering: "seq_cst" });
// Generates: fence seq_cst
```

Chapter 8

Memory Model and Address Spaces

8.1 Stack vs Heap

LLVM IR supports two memory allocation strategies:

Stack allocation (alloca):

```
const alloc = new AllocaInst({
  name: "local_var",
  allocatedType: i32,
  align: 4
});
// Generates: %local_var = alloca i32, align 4
```

- Automatically deallocated when function returns
- Fast (stack pointer bump)
- Limited size (typical stack: 1-8 MB)
- Good for small, short-lived data

Heap allocation (external malloc):

```
// Declare malloc
const mallocDecl = new LLVMFunction({
  name: "malloc",
  returnType: ptr,
  isExternal: true
});
mallocDecl.addParameter(new Parameter({ type: i64, name: "size" }));

// Call malloc
const heapPtr = new CallInst({
  name: "heap_ptr",
  returnType: ptr,
  functionName: "malloc",
  args: [{ type: i64, value: 4096 }]
});
// Generates: %heap_ptr = call ptr @malloc(i64 4096)
```

- Manual deallocation required (`free`)
- Slower (OS allocator overhead)
- Virtually unlimited size (limited by RAM)
- Good for large, long-lived data

8.2 Alignment

Alignment specifies byte boundaries for memory access:

```
const load = new LoadInst({
  name: "val",
  type: i64,
  pointerType: ptr,
  pointer: "ptr",
  align: 8  // 8-byte alignment for 64-bit integers
});
// Generates: %val = load i64, ptr %ptr, align 8
```

Why alignment matters:

- **x86**: Misaligned access is slow (extra memory cycles)
- **ARM**: Misaligned access may trap (SIGBUS crash)
- **GPU**: Coalesced memory access requires alignment

Common alignments:

Type	Alignment	Reason
`i8`	1 byte	Single byte, no alignment needed
`i16`	2 bytes	16-bit values must be 2-byte aligned
`i32`	4 bytes	32-bit values must be 4-byte aligned
`i64`	8 bytes	64-bit values must be 8-byte aligned
`float`	4 bytes	IEEE 754 single precision (32-bit)
`double`	8 bytes	IEEE 754 double precision (64-bit)
`ptr`	8 bytes (64-bit)	Pointer size depends on architecture
Struct	Largest member	Structs align to their largest field

8.3 Address Spaces (GPU Memory)

GPUs use **address spaces** to distinguish between memory types:

Space	Name	Scope	Performance
0	Generic	All	Compiler chooses
1	Global	All threads	Slow (DRAM)
3	Shared (CUDA/AMD)	Thread block	Fast (on-chip)
4	Constant	All threads	Fast (cached)
5	Local (CUDA)	Single thread	Varies (register spill)

Declaring global memory (address space 1):

```
import { GlobalVariable } from "@euriklis/llvm-ir";

const globalArray = new GlobalVariable({
  name: "global_data",
  type: new ArrayType({ elementType: float, size: 1024 }),
  addressSpace: 1,  // Global memory
  linkage: "external"
});
// Generates: @global_data = external addrspace(1) global [1024 x float
   ]
```

Declaring shared memory (address space 3):

```
const sharedMem = new GlobalVariable({
  name: "shared_tile",
  type: new ArrayType({ elementType: float, size: 256 }),
  addressSpace: 3,  // Shared memory (CUDA __shared__)
  linkage: "internal"
});
// Generates: @shared_tile = internal addrspace(3) global [256 x float]
```

8.4 GetElementPtr (GEP) Operations

`GetElementPtr` computes addresses within aggregates (arrays, structs):

```
// Array indexing: array[idx]
const gep = new GetElementPtrInst({
  name: "elem_ptr",
  baseType: i32,
  ptrType: ptr,
  pointer: "array",
  indices: [{ type: i32, value: "idx" }]
});
// Generates: %elem_ptr = getelementptr i32, ptr %array, i32 %idx

// Struct field access: struct.field1
const structGep = new GetElementPtrInst({
  name: "field_ptr",
  baseType: structType,
  ptrType: ptr,
  pointer: "struct_ptr",
  indices: [
    { type: i32, value: 0 },   // First: dereference pointer
    { type: i32, value: 1 }    // Second: select field 1
  ]
});
// Generates: %field_ptr = getelementptr %StructType, ptr %struct_ptr,
   i32 0, i32 1
```

Important: GEP does *not* load memory—it only computes addresses. Use `LoadInst` to read the value.

8.5 Volatile Access

Mark memory operations as `volatile` to prevent optimization:

```
const load = new LoadInst({
  name: "val",
  type: i32,
  pointerType: ptr,
  pointer: "mmio_reg",
  align: 4,
  isVolatile: true  // Prevent optimization
});
// Generates: %val = load volatile i32, ptr %mmio_reg, align 4
```

Use cases:

- Memory-mapped I/O registers
- Signal handlers
- Multithreaded shared variables (prefer atomics instead)

8.6 Atomic Operations

For thread-safe memory access, use atomic instructions:

```
const atomicLoad = new LoadInst({
  name: "val",
  type: i32,
  pointerType: ptr,
  pointer: "shared_var",
  align: 4,
  atomic: true,
  ordering: "acquire"
});
// Generates: %val = load atomic i32, ptr %shared_var acquire, align 4

const atomicStore = new StoreInst({
  valueType: i32,
  value: 42,
  pointerType: ptr,
  pointer: "shared_var",
  align: 4,
  atomic: true,
  ordering: "release"
});
// Generates: store atomic i32 42, ptr %shared_var release, align 4
```

Memory orderings:

- `unordered`: No synchronization (fastest)
- `monotonic`: Single total order (still weak)
- `acquire`: Synchronize-with release (for loads)
- `release`: Synchronize-with acquire (for stores)
- `acq_rel`: Both acquire and release
- `seq_cst`: Sequentially consistent (safest, slowest)

Chapter 9

Type System Reference

9.1 Introduction

LLVM IR is strongly typed. Every value, instruction, and function has an explicit type.

9.2 Integer Types

Fixed-width integers:

```
import { IntType } from "@euriklis/llvm-ir";

const i1 = LLVMCodegen.types.i1;      // Boolean (1 bit)
const i8 = LLVMCodegen.types.i8;      // Byte (8 bits)
const i16 = LLVMCodegen.types.i16;    // Short (16 bits)
const i32 = LLVMCodegen.types.i32;    // Int (32 bits)
const i64 = LLVMCodegen.types.i64;    // Long (64 bits)
const i128 = LLVMCodegen.types.i128;  // Quad (128 bits)
```

Arbitrary-width integers:

```
const i7 = new IntType({ bits: 7 });    // 7-bit integer
const i24 = new IntType({ bits: 24 });  // 24-bit integer (RGB color)
const i256 = new IntType({ bits: 256 }); // 256-bit integer (crypto)
// Generates: i7, i24, i256
```

LLVM supports integers from `i1` to `i2^23-1` (8 million bits).

9.3 Floating-Point Types

```
import { FloatType } from "@euriklis/llvm-ir";

const half = new FloatType("half");      // f16: 16-bit half precision
const bfloat = new FloatType("bfloat");  // bf16: Brain float (ML)
const float = new FloatType("float");    // f32: Single precision (IEEE
    754)
const double = new FloatType("double");  // f64: Double precision
const fp128 = new FloatType("fp128");    // Quad precision (128-bit)
```

When to use each:

- `half (f16)`: GPU compute, ML inference (low precision, high throughput)
- `bfloat (bf16)`: ML training (Google TPUs, NVIDIA Ampere+)
- `float (f32)`: Default for graphics and general compute

- `double (f64)`: Scientific computing, high-precision math
- `fp128`: Rare (extreme precision needs)

9.4 Pointer Types

In LLVM IR, pointers are opaque (no element type in modern LLVM):

```
const ptr = LLVMCodegen.types.ptr;  // Opaque pointer
// Generates: ptr

// Address space variant
const ptr_addrspace3 = new PointerType(3);  // ptr addrspace(3)
// Generates: ptr addrspace(3) (CUDA shared memory)
```

9.5 Array Types

```
import { ArrayType } from "@euriklis/llvm-ir";

const intArray = new ArrayType({
  elementType: i32,
  size: 10
});
// Generates: [10 x i32]

const floatMatrix = new ArrayType({
  elementType: new ArrayType({ elementType: float, size: 4 }),
  size: 4
});
// Generates: [4 x [4 x float]] (4x4 matrix)
```

9.6 Struct Types

```
import { StructType } from "@euriklis/llvm-ir";

// Named struct
const Point = new StructType({
  name: "Point",
  fields: [
    { type: float },
    { type: float },
    { type: float }
  ]
});
// Generates: %Point = type { float, float, float }

// Packed struct (no padding)
const PackedPoint = new StructType({
  name: "PackedPoint",
  fields: [
    { type: i8 },
    { type: i32 }
  ],
  packed: true
```

```
});
// Generates: %PackedPoint = type <{ i8, i32 }>
```

9.7 Vector Types

SIMD vectors for parallel operations:

```
import { VectorType } from "@euriklis/llvm-ir";

// <4 x float> - 4-element float vector (SSE, NEON)
const vec4 = new VectorType({
  elementType: float,
  count: 4
});

// <8 x i32> - 8-element integer vector (AVX)
const vec8i = new VectorType({
  elementType: i32,
  count: 8
});

// Vector addition
const vadd = new BinaryInst({
  name: "vsum",
  opcode: LLVMCodegen.opcodes.fadd,
  type: vec4,
  lhs: "a",
  rhs: "b"
});
// Generates: %vsum = fadd <4 x float> %a, %b
// This adds 4 floats in parallel!
```

9.8 Function Types

```
import { FunctionType } from "@euriklis/llvm-ir";

// int add(int, int)
const addFnType = new FunctionType({
  returnType: i32,
  paramTypes: [i32, i32]
});
// Type: i32 (i32, i32)

// void print(char*)
const printFnType = new FunctionType({
  returnType: voidType,
  paramTypes: [ptr]
});
// Type: void (ptr)

// Variadic: int printf(char*, ...)
const printfFnType = new FunctionType({
  returnType: i32,
  paramTypes: [ptr],
  isVarArg: true
```

```
});
// Type: i32 (ptr, ...)
```

9.9 Void Type

Special type for functions that don't return a value:

```
const { void: voidType } = LLVMCodegen.types;

const func = new LLVMFunction({
  name: "doSomething",
  returnType: voidType
});
// Generates: define void @doSomething() { ... }
```

9.10 Type Compatibility

LLVM is strictly typed. You cannot mix types without explicit conversion:

```
// ERROR: Cannot add i32 and i64
const wrong = new BinaryInst({
  opcode: "add",
  type: i32,
  lhs: "x",  // i32
  rhs: "y"   // i64 - type mismatch!
});

// CORRECT: Convert first
const y64 = new ZExtInst({
  name: "y_extended",
  from: "y",
  fromType: i32,
  toType: i64
});
const correct = new BinaryInst({
  opcode: "add",
  type: i64,
  lhs: y64,
  rhs: "x"
});
```

Part V

Advanced Topics

Chapter 10

GPU Memory Hierarchies and Thread Synchronization

10.1 The GPU Memory Hierarchy

GPU programming requires understanding the memory hierarchy and synchronization primitives to achieve high performance.

GPUs have a complex memory hierarchy with vastly different performance characteristics:

Memory Type	Latency	Bandwidth	Use Case
Registers	1 cycle	>10 TB/s	Loop counters, temporaries
Shared/LDS	20-30 cycles	1-2 TB/s	Tile caching, reductions
L1 Cache	50-100 cycles	500 GB/s - 1 TB/s	Automatic caching
L2 Cache	100-200 cycles	200-500 GB/s	Automatic caching
Global/VRAM	200-400 cycles	500 GB/s - 2 TB/s	Large datasets

The key to GPU performance is **minimizing global memory access** by using registers and shared memory.

10.2 Global Memory (Address Space 1)

Global memory is large (GBs) but slow (hundreds of cycles latency):

```
import { GlobalVariable, ArrayType } from "@euriklis/llvm-ir";

const globalData = new GlobalVariable({
  name: "large_array",
  type: new ArrayType({ elementType: float, size: 1048576 }),  // 1M
     floats
  addressSpace: 1,  // Global memory
  linkage: "external"
});
// Generates: @large_array = external addrspace(1) global [1048576 x
   float]
```

Access patterns matter:

- **Coalesced access**: Threads access consecutive addresses → 1 memory transaction
- **Strided access**: Threads access every Nth element → multiple transactions
- **Random access**: Worst case → many transactions

10.3 Shared Memory (Address Space 3)

Shared memory is small (KB) but fast (20-30 cycle latency):

```
const sharedTile = new GlobalVariable({
  name: "tile",
  type: new ArrayType({ elementType: float, size: 256 }),  // 256
     floats (1 KB)
  addressSpace: 3,  // Shared memory
  linkage: "internal"
});
// Generates: @tile = internal addrspace(3) global [256 x float]
```

Why shared memory is 100x faster:

- On-chip SRAM (not external DRAM)
- Shared across threads in the same block
- Perfect for tiling algorithms (reuse data)

10.4 Constant Memory (Address Space 4)

Constant memory is read-only and cached:

```
const constants = new GlobalVariable({
  name: "coefficients",
  type: new ArrayType({ elementType: float, size: 64 }),
  addressSpace: 4,  // Constant memory
  linkage: "internal",
  isConstant: true
});
// Generates: @coefficients = internal addrspace(4) constant [64 x
   float]
```

When to use:

- Filter coefficients (convolution)
- Lookup tables
- Configuration parameters

10.5 Thread Synchronization

When using shared memory, threads must synchronize to avoid race conditions.

Barrier synchronization:

```
import { CallInst } from "@euriklis/llvm-ir";

// CUDA: __syncthreads()
const barrier = new CallInst({
  functionName: "llvm.nvvm.barrier0",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.nvvm.barrier0()

// AMD ROCm: barrier()
const barrierAMD = new CallInst({
  functionName: "llvm.amdgcn.s.barrier",
```

```
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.amdgcn.s.barrier()

// OpenCL: barrier(CLK_LOCAL_MEM_FENCE)
const barrierOCL = new CallInst({
  functionName: "_Z7barrierj",  // OpenCL barrier
  returnType: voidType,
  args: [{ type: i32, value: 1 }]  // CLK_LOCAL_MEM_FENCE
});
```

Package-provided helper functions:

Instead of manually creating `CallInst` objects, the package provides convenient helper methods:

```
import { NVVMCodegen, AMDGPUCodegen, SPIRVCodegen } from "@euriklis/
    llvm-ir";

// CUDA: Use the helper (recommended)
const barrier = NVVMCodegen.cuda.syncThreads();
// Returns a CallInst configured for llvm.nvvm.barrier0()

// AMD ROCm: Use the helper
const barrierAMD = AMDGPUCodegen.rocm.barrier();
// Returns a CallInst for llvm.amdgcn.s.barrier()

// OpenCL: Use the helper
const barrierOCL = SPIRVCodegen.opencl.barrier();
// Returns a CallInst for _Z7barrierj with correct arguments

// All helpers return CallInst objects ready to add to your basic
    blocks
entry.addInstruction(barrier);
```

These helpers ensure correct intrinsic names and arguments, making your code cleaner and less error-prone.

What barriers do:

1. All threads in the block wait at the barrier
2. No thread proceeds until ALL threads arrive
3. Ensures memory operations before barrier are visible after

Critical rule: Barriers must be in uniform control flow—all threads must execute the same barrier, or deadlock occurs.

```
// WRONG: Conditional barrier (DEADLOCK!)
if (threadIdx.x < 16) {
  barrier();  // Only some threads hit barrier!
}

// CORRECT: All threads hit barrier
barrier();  // Everyone waits
if (threadIdx.x < 16) {
  // Now safe to use synchronized data
}
```

10.6 Memory Fences

What are memory fences?

In JavaScript/TypeScript, when you write `array[i] = 42;`, it happens immediately and in order. But in GPUs and CPUs, the hardware can reorder memory operations for performance. A **memory fence** is like saying "wait, make sure all pending writes are actually visible before continuing."

Barriers vs. Fences:

Think of them like this:

- **Barrier** (`syncThreads()`): "Everyone stop and wait here. Don't proceed until all threads arrive." (Synchronization + memory ordering)
- **Fence** (`threadfence()`): "Make my writes visible, but don't wait for other threads." (Memory ordering only)

When to use fences:

Use fences for lock-free algorithms and atomic operations where you need memory ordering guarantees but don't want the overhead of waiting for all threads.

Example: GPU memory fences

```
import { FenceInst, CallInst } from "@euriklis/llvm-ir";

// Standard LLVM fence (works on all architectures)
const fence = new FenceInst({
  ordering: "seq_cst"  // Sequential consistency (strongest)
});
// Generates: fence seq_cst

// CUDA threadfence (block-level) - ensures writes visible to block
const fenceBlock = new CallInst({
  functionName: "llvm.nvvm.membar.cta",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.nvvm.membar.cta()

// CUDA threadfence (global-level) - ensures writes visible globally
const fenceGlobal = new CallInst({
  functionName: "llvm.nvvm.membar.gl",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.nvvm.membar.gl()
```

Memory ordering levels:

`FenceInst` supports different ordering strengths (from weakest to strongest):

- `"acquire"`: Ensures reads after fence see all writes before
- `"release"`: Ensures writes before fence are visible after
- `"acq_rel"`: Both acquire and release
- `"seq_cst"`: Sequentially consistent (default, safest)

For most use cases, use `"seq_cst"` (sequential consistency). Only use weaker orderings if you understand low-level memory models and need the performance.

10.7 Tiled Matrix Multiplication Example

Now let's see why shared memory matters with a real example: matrix multiplication $C = A \times B$.

Naive version (global memory only):

Each thread computes one element of C:

```
for (int k = 0; k < N; k++) {
  sum += A[row][k] * B[k][col];  // 2N global memory accesses per
     thread
}
```

Problem: **Every iteration reads from global memory** (200-400 cycles each). For $N = 1024$, that's 2048 slow memory accesses per thread.

Tiled version (shared memory):

1. Load a **tile** of A and B into shared memory
2. Synchronize threads (barrier)
3. Compute using fast shared memory
4. Repeat for next tile

```
import { NVVMCodegen, GlobalVariable, ArrayType } from "@euriklis/llvm-
   ir";

const TILE_SIZE = 16;

// Declare shared memory tiles (address space 3)
const tileA = new GlobalVariable({
  name: "tileA",
  type: new ArrayType({
    elementType: new ArrayType({ elementType: float, size: TILE_SIZE })
       ,
    size: TILE_SIZE
  }),
  addressSpace: 3,  // Shared memory
  linkage: "internal"
});

const tileB = new GlobalVariable({
  name: "tileB",
  type: new ArrayType({
    elementType: new ArrayType({ elementType: float, size: TILE_SIZE })
       ,
    size: TILE_SIZE
  }),
  addressSpace: 3,
  linkage: "internal"
});

// Kernel structure (simplified):
// 1. Load tile from A (global) into tileA (shared)
// 2. Load tile from B (global) into tileB (shared)
// 3. barrier() - ensure all loads complete
// 4. Compute using tileA and tileB (fast!)
// 5. barrier() - before loading next tile
// 6. Repeat for all tiles
```

Full implementation with all details (thread indexing, tile loop, PHI nodes) is available in the repository examples.

Performance analysis:
Naive (global only):

- Each thread: $2N$ global loads (400 cycles each)
- Total latency: $\sim 800N$ cycles
- For $N = 1024$: $\sim 819,200$ cycles

Tiled (shared memory):

- Each tile: $TILE\_SIZE^2$ global loads shared across block
- Tile reused $TILE\_SIZE$ times from shared (20 cycles each)
- Total latency: $\sim \frac{800N}{TILE\_SIZE} + 20N$
- For $N = 1024, TILE = 16$: $\sim 71,680$ cycles
- **11x speedup!**

10.8 Key Takeaways

1. **Use shared memory** to cache frequently accessed data
2. **Synchronize with barriers** after loading and before computing
3. **Tile your algorithms** to fit working sets in shared memory
4. **Coalesce global accesses** when loading tiles
5. **Avoid bank conflicts** in shared memory (advanced topic)

The tiled pattern applies to many GPU algorithms: convolutions, FFTs, reductions, sorting.

Chapter 11

Future Work: Apple Silicon GPU Support

11.1 Current State

This package currently supports Apple Silicon **CPUs** via the ARM/AArch64 backend:

```
import { ARMCodegen } from "@euriklis/llvm-ir";

// Target Apple M3/M5 CPU cores
const codegen = new ARMCodegen("apple_module", true);  // 64-bit
codegen.targetTriple = "arm64-apple-macosx";

// Use NEON SIMD for vectorization (128-bit vectors)
const vec4 = new VectorType({ elementType: float, count: 4 });
// Generates efficient ARM NEON instructions
```

This works well for CPU-bound computations with SIMD acceleration.

11.2 The Apple GPU Challenge

Apple M3/M5 chips have powerful GPUs that are excellent for AI and parallel computing. However, unlike NVIDIA and AMD, Apple uses a **proprietary GPU architecture**:

How GPU compilation works:

- **NVIDIA**: Your code → LLVM IR (NVPTX) → PTX → CUDA binary (supported)
- **AMD**: Your code → LLVM IR (AMDGPU) → GCN/RDNA binary (supported)
- **Apple**: Your code → Metal Shading Language[8] → **AIR (proprietary)** → Metal binary (not supported)

The problem: AIR is based on LLVM IR, but Apple keeps the format **closed and undocumented**[18]. We cannot generate it directly.

11.3 Why This Matters for TypeScript Developers

If you're coming from a JavaScript/TypeScript background, here's an analogy:

- CUDA/AMD are like **open APIs**—you can generate code programmatically (what this package does)
- Metal is like a **proprietary framework**—you must use Apple's tools and pre-built libraries

Think of it like this:

```
// CUDA/AMD (what this package does)
const kernel = generateLLVMIR({
  instructions: [...],  // Full programmatic control
  intrinsics: [...]
});

// Metal (what Apple requires)
const kernel = `
  kernel void myKernel(...) {
    // Must write Metal Shading Language as text
  }
`;
// Then compile with Apple's tools
```

11.4 How PyTorch and TensorFlow Do It

Frameworks like PyTorch don't generate Metal code either. Instead, they use **MPS**[13][5]—Apple's high-level library:

- Apple provides hundreds of **pre-optimized GPU kernels** (matrix multiply, convolution, etc.)[6]
- PyTorch maps operations to these kernels via the **MPSGraph API**
- This works great for standard ML operations
- For custom kernels, you must write `.metal` files by hand[8]

11.5 What We're Waiting For

Apple has two paths to enable this package's Metal support:

Option 1: Open the AIR format specification

If Apple documents how AIR works[18], we could:

```
import { MetalCodegen } from "@euriklis/llvm-ir";

const codegen = new MetalCodegen("my_kernel");
codegen.targetTriple = "air64-apple-macosx";
// Generate AIR directly, just like we do for NVPTX
```

Option 2: Stable Metal Shading Language generation

Alternatively, we could generate `.metal` source files programmatically[7]:

```
const codegen = new MetalCodegen("my_kernel");
const metalSource = codegen.toMetalSource();  // Generate .metal text
// User compiles: xcrun metal -c kernel.metal
```

This is less elegant than LLVM IR generation, but would work.

11.6 Current Workarounds

Until Apple opens AIR or provides better tooling, TypeScript/JavaScript developers targeting Apple GPUs should:

1. **Use high-level frameworks**: PyTorch MPS, TensorFlow, Core ML
2. **Write Metal Shading Language directly**: Create `.metal` files for custom kernels

3. **Use this package for CPU SIMD**: ARM/NEON support works today
4. **Prototype on NVIDIA/AMD**: Develop with this package's GPU support, deploy to Apple via Metal ports

11.7 Unified Memory Advantage

One unique strength of Apple Silicon: **unified memory architecture**.

- CPU and GPU share the same memory pool (16-128 GB)
- No expensive CPU↔GPU copies (unlike NVIDIA)
- Ideal for workflows mixing CPU and GPU work

Once Metal support is added, this will make Apple Silicon extremely competitive for heterogeneous computing workflows.

11.8 Timeline

When AIR becomes public or Apple provides stable LLVM IR → Metal tooling, we will add:

- `MetalCodegen` class matching the API of `NVVMCodegen`, `AMDGPUCodegen`
- Metal-specific intrinsics (`threadgroup` memory, barriers, SIMD-groups)
- Complete examples for Apple GPU programming
- Validation with Apple's Metal compiler

We're monitoring Apple's developer documentation and LLVM commits for any changes. If you have insights into Metal's IR generation, please contribute!

11.9 Community Contributions Welcome

Possible paths forward:

- **Metal Shading Language generator**: Generate `.metal` source text (similar to how we generate LLVM IR text)
- **MPSGraph bindings**: High-level TypeScript API for Metal Performance Shaders
- **AIR reverse-engineering**: Document the format (challenging, legal concerns)
- **Lobby Apple**: Request official AIR documentation or LLVM Metal backend

If you're interested in helping, see the repository's contribution guidelines.

Bibliography

[1] *Efficient Algorithms for the Uniform Tokenization Problem*. Proceedings of the ACM on Programming Languages, 2024. Available at: https://dl.acm.org/doi/10.1145/3720498

[2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools, Second Edition*. Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.

[3] Andrew W. Appel. *Modern Compiler Implementation in ML/Java/C*. Cambridge University Press, 2004. ISBN: 978-0-521-60765-0.

[4] Michael J. Fischer and Michael S. Paterson. *String Matching and Other Products*. In Richard M. Karp (Ed.), *Complexity of Computation*, SIAM-AMS Proceedings, Vol. 7, pp. 113–125, 1974. Foundational work on FFT-based pattern matching algorithms using polynomial representations.

[5] Apple Inc. *Accelerated PyTorch training on Mac*. Apple Developer, 2024. Available at: https://developer.apple.com/metal/pytorch/

[6] Apple Inc. *Metal Documentation*. Apple Developer, 2024. Available at: https://developer.apple.com/metal/

[7] Apple Inc. *Metal Programming Guide*. Apple Developer Documentation Archive, 2024. Available at: https://developer.apple.com/library/archive/documentation/Miscellaneous/Conceptual/MetalProgrammingGuide/

[8] Apple Inc. *Metal Shading Language Specification, Version 4*. Apple Developer Documentation, 2025. Available at: https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf

[9] Bruno Cardoso Lopes and Rafael Auler. *Getting Started with LLVM Core Libraries*. Packt Publishing, 2014. ISBN: 978-1-78216-692-4. A comprehensive guide to LLVM infrastructure covering compiler design, IR generation, optimization passes, and code generation for multiple architectures.

[10] ApsarasX. *llvm-bindings: LLVM bindings for Node.js/JavaScript/TypeScript*. GitHub, 2021. Available at: https://github.com/ApsarasX/llvm-bindings

[11] Cornell Capra. *node-llvmc: JavaScript/TypeScript FFI bindings for the LLVM C API*. GitHub, 2023. Available at: https://github.com/cucapra/node-llvmc

[12] Keith D. Cooper and Linda Torczon. *Engineering a Compiler, Third Edition*. Morgan Kaufmann (Elsevier), 2023. ISBN: 978-0-12-815412-0.

[13] PyTorch Contributors. *MPS backend — PyTorch Documentation*. PyTorch, 2024. Available at: https://docs.pytorch.org/docs/stable/notes/mps.html

[14] Leo A. Meyerovich and Ariel S. Rabkin. *Empirical Analysis of Programming Language Adoption*. Proceedings of the ACM international conference on Object oriented programming systems languages and applications (OOPSLA), 2013. Available at: https://doi.org/10.1145/2509136.2509515

[15] Maël Nison et al. *llvm-node: Node.js bindings for LLVM*. npm package, 2023. Available at: https://www.npmjs.com/package/llvm-node

[16] NVIDIA Corporation. *NVVM IR Specification 13.1*. NVIDIA CUDA Documentation, 2025. Available at: https://docs.nvidia.com/cuda/nvvm-ir-spec/index.html. Defines the NVVM intermediate representation dialect of LLVM IR for NVIDIA GPU programming.

[17] David A. Patterson and John L. Hennessy. *Computer Organization and Design, Fifth Edition: The Hardware/Software Interface*. Morgan Kaufmann (Elsevier), 2013. ISBN: 978-0-12-407726-3.

[18] SamoZ256. *Breaking down Metal's intermediate representation format*. Medium, 2024. Available at: https://medium.com/@samuliak/breaking-down-metals-intermediate-representation-format-41827022489c

[19] The Khronos Group. *SPIR-V Specification*. Khronos Group Standards, 2024. Available at: https://www.khronos.org/spir/. Standard Portable Intermediate Representation for Vulkan and OpenCL.

[20] Suyog Sarda, Mayur Pandey. *LLVM Essentials*. Packt Publishing, 2015. An excellent overview of LLVM fundamentals; this handbook extends those concepts to the TypeScript ecosystem and multi-architecture workflows.

[21] The LLVM Project. *LLVM Language Reference Manual*. LLVM Documentation, 2025. Available at: https://llvm.org/docs/LangRef.html. The authoritative specification of LLVM IR syntax, semantics, intrinsics, and data layout string format.

[22] Wikipedia Contributors. *LLVM*. Wikipedia, The Free Encyclopedia, 2025. Available at: https://en.wikipedia.org/wiki/LLVM. Comprehensive overview of LLVM project history, supported architectures, and backend implementations.

About This Handbook

This comprehensive handbook bridges the gap between high-level TypeScript programming and low-level LLVM IR code generation. The main goal of this project is to create a TypeScript-supported LLVM generator designed for computationally intensive applications, including heterogeneous programming and GPU-accelerated computing. This project will be integrated into more advanced packages of the @euriklis family, providing a robust way for heterogeneous programming via TypeScript. Whether you're building compilers, optimizing performance-critical applications, or targeting multiple architectures, this guide provides practical knowledge for leveraging LLVM's powerful infrastructure.

Main topics:

- Core concepts of LLVM IR and its type system
- Cross-platform compilation for ARM, x86, and GPU architectures
- GPU programming with NVIDIA NVVM, AMD, and SPIR-V
- Memory management and optimization techniques
- Complete instruction reference and practical examples

About the Author

Velislav Karastoychev is an economist and the founder of Euriklis LTD and co-founder of One Agent LTD. With rich experience in econometrics and the implementation of statistical test hypothesis algorithms, he has translated and implemented in TypeScript a vast collection of algorithms for mathematical programming, optimization, linear algebra, and graph/network analysis. His goal is to elevate TypeScript to the level of a comprehensive language for data processing, analysis, and machine learning, bringing the power of low-level optimization to high-level application development.

GitHub Repository:

https://github.com/VelislavKarastoychev/euriklis-llvm-ir